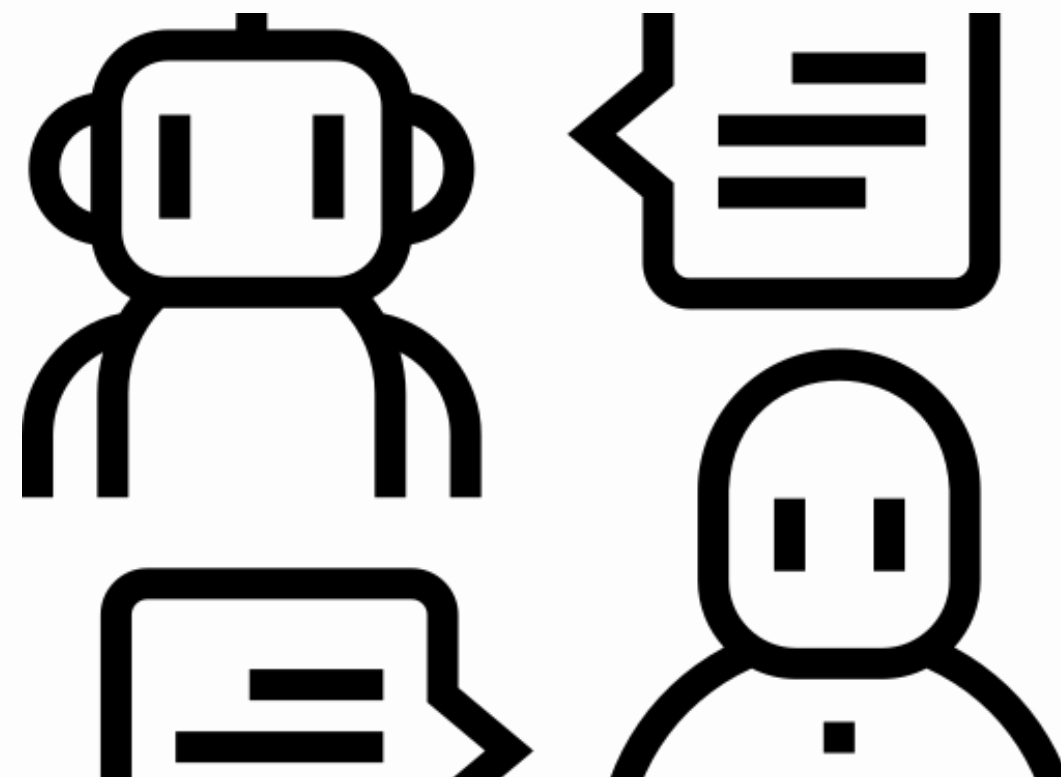


Proposta para o Trabalho de Conclusão de Curso



TUTOR DE IA PARA O APRENDIZADO DE INGLÊS

ÍNDICE

01 | Introdução e Justificativa

02 | Fundamentação teórica

03 | Proposta e Objetivos

04 | Materiais e métodos

05 | Resultados

06 | Conclusão

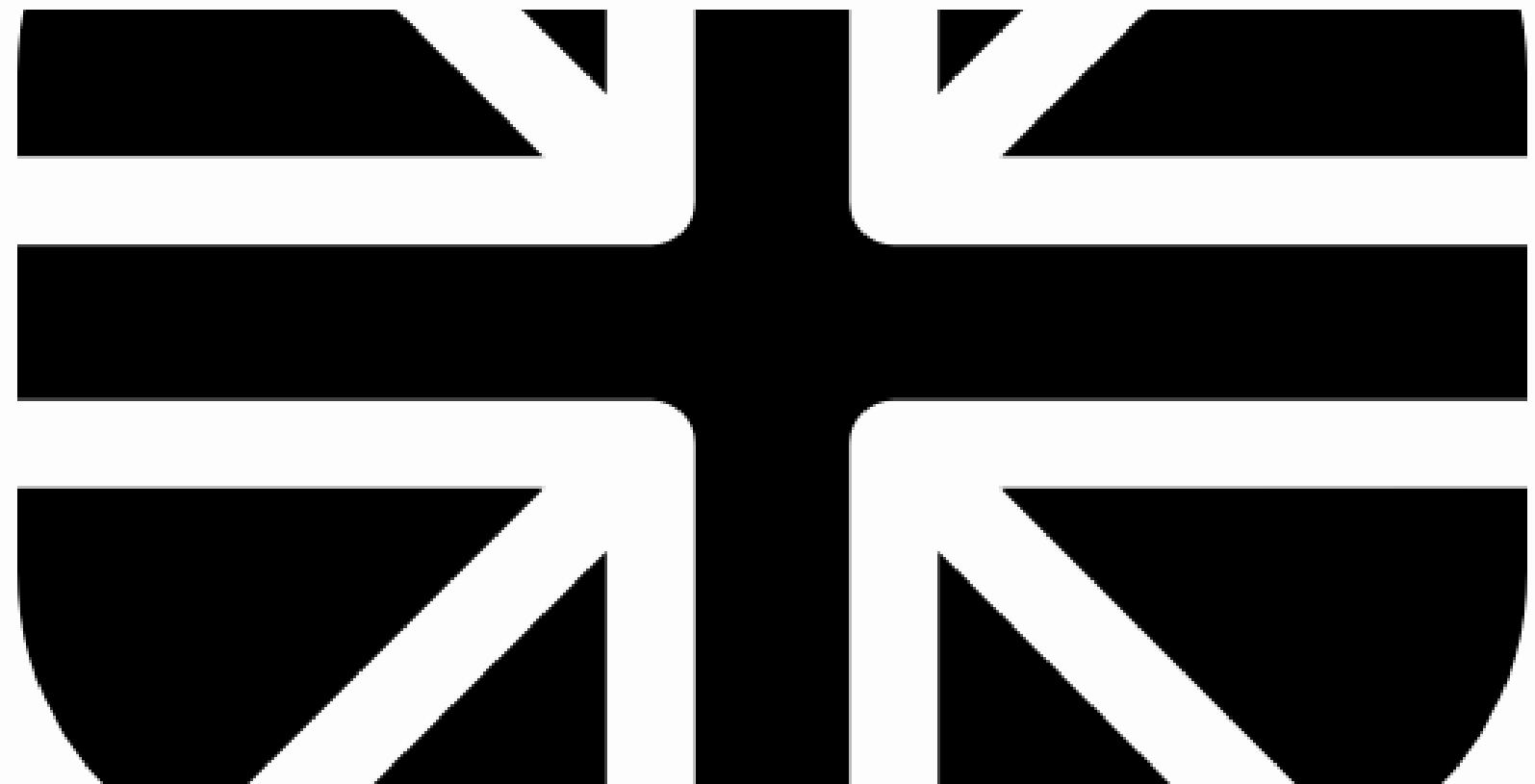
INTRODUÇÃO E JUSTIFICATIVA

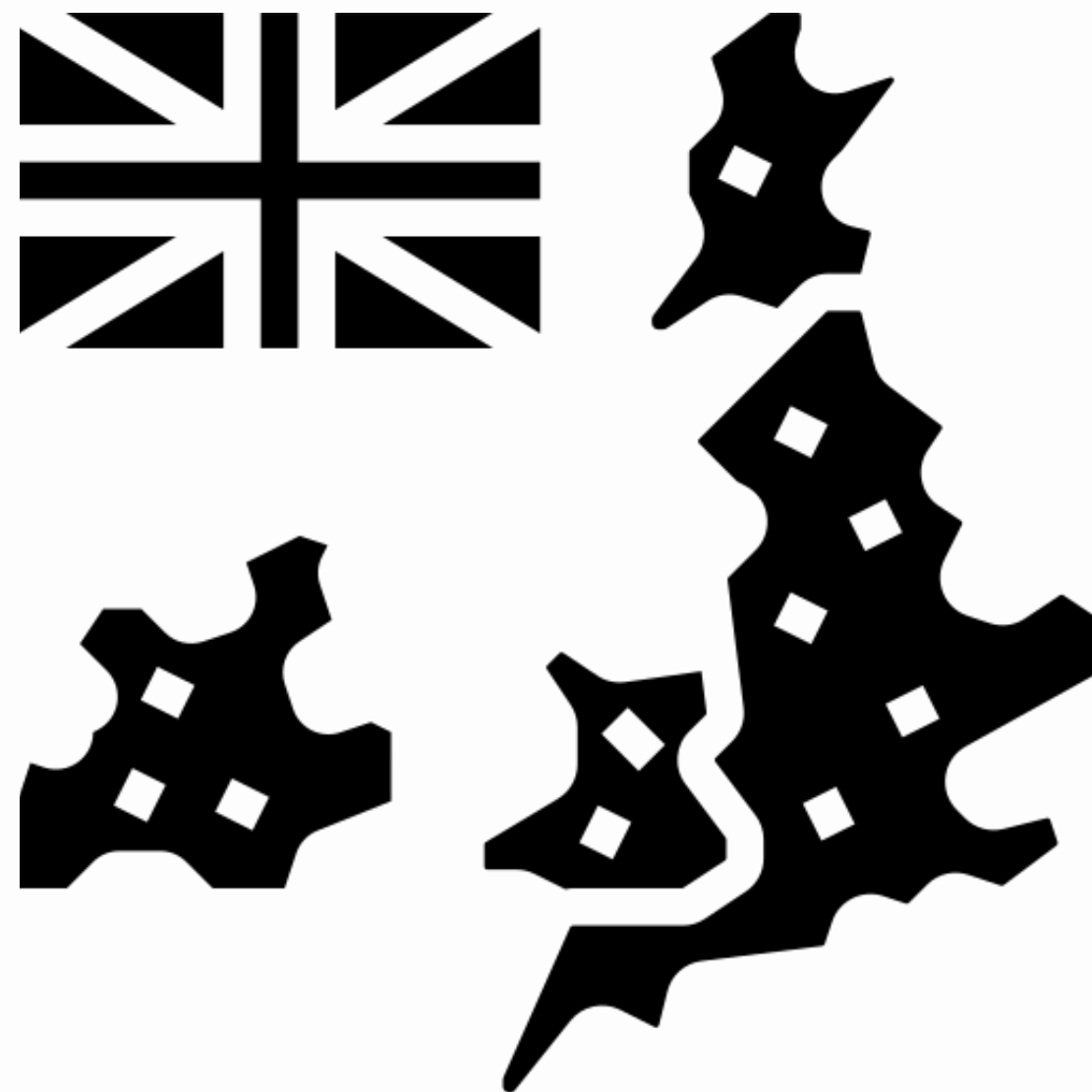
O INGLÊS GLOBALIZADO.



01 **Acesso à informação:** O conteúdo especializado com informação de qualidade normalmente está em inglês (Ilysovna, 2020).

02 **Diferencial competitivo no mercado:** Os profissionais que possuem o inglês como segunda língua possuem salários maiores (Ilysovna, 2020).





AS BARREIRAS LINGÜÍSTICAS

Abordagens tradicionais gratuitas: Frequentemente focadas em gramática, com pouca ênfase na comunicação;

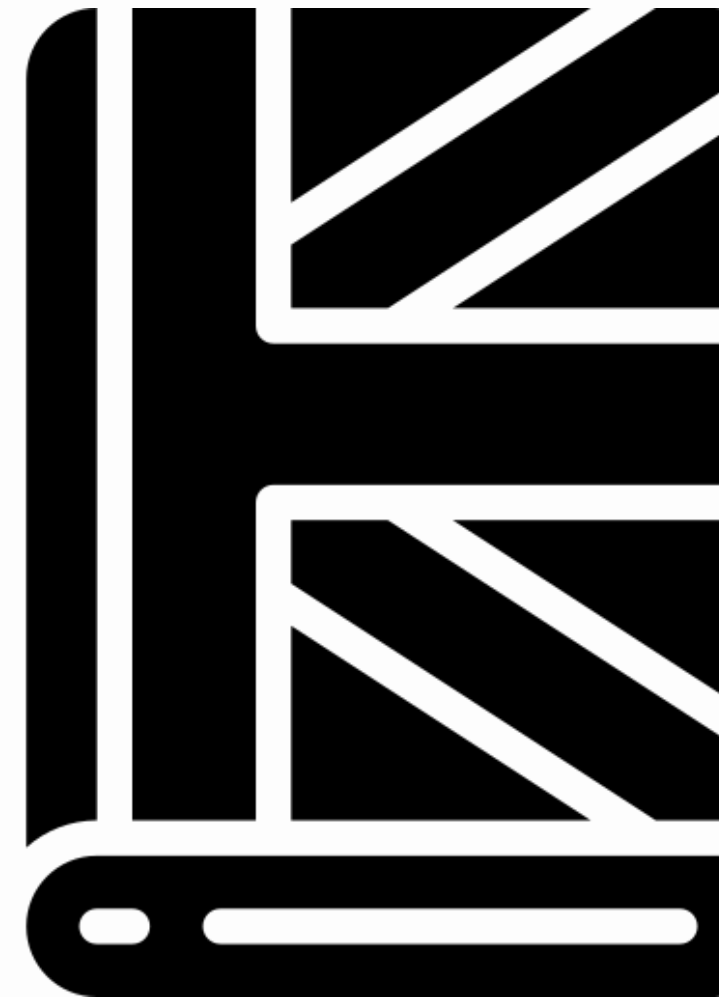
Custos elevados: Escolas de idiomas possuem valores inacessíveis para uma ampla parcela da população;

Conversação: Limitações de acesso a falantes nativos para prática conversacional .

A BARREIRA CRÍTICA

Ansiedade conversacional

Horwitz et al. (1986), identificam a ansiedade como um obstáculo significativo na prática de conversação. O medo de cometer erros e do julgamento social frequentemente inibe a participação do aprendiz.



A EVOLUÇÃO DA TECNOLOGIA NO APRENDIZADO DE IDIOMAS

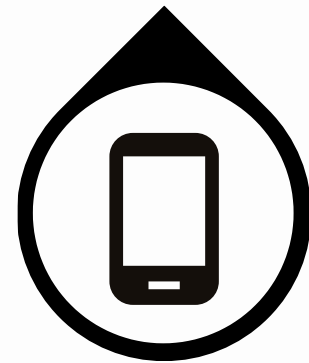
MALL

(*Mobile-Assisted Language Learning*). (Shortt et al., 2023)
Com a popularização dos dispositivos móveis, 98,8% dos brasileiros acessam internet por celulares (Ibge, 2024).



CALL

(*Computer-Assisted Language Learning*) dos anos 1960 (Levy, 1997).



TUTORES DE IA

Nova geração de sistemas com a ascensão dos LLMs (Lyu; Lai; Guo, 2025), que buscam oferecer conversação natural e *feedback* instantâneo (Ex: Duolingo MAX, Praktika).

IMPACTO PEDAGÓGICO DOS TUTORES DE IA

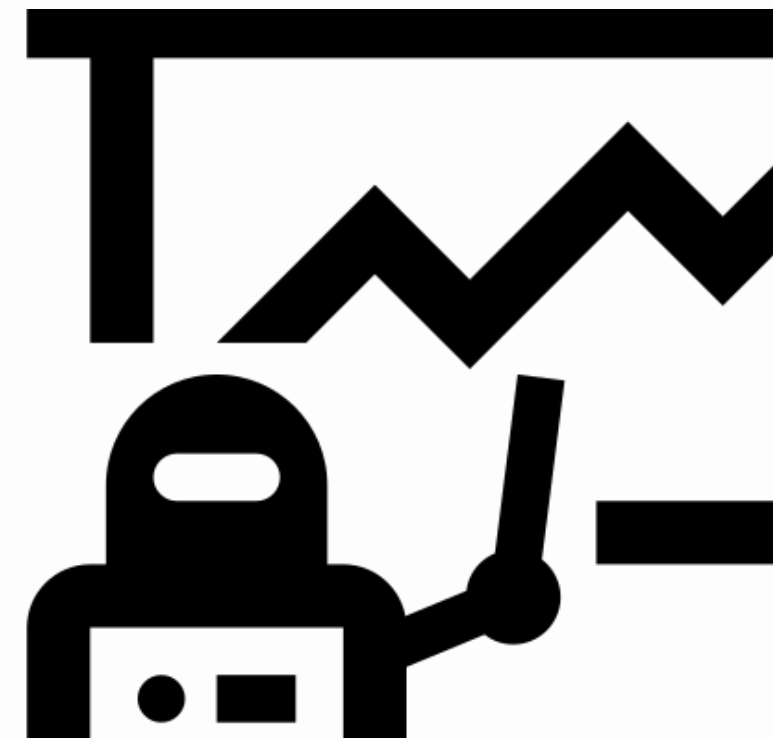
Os Tutores de IA demonstram superioridade promissora em relação aos métodos anteriores.

Mitigação da ansiedade conversacional

Oferecem um ambiente seguro para a prática conversacional, sem receio de julgamento social (Zhang; Huang, 2024).

Personalização do aprendizado

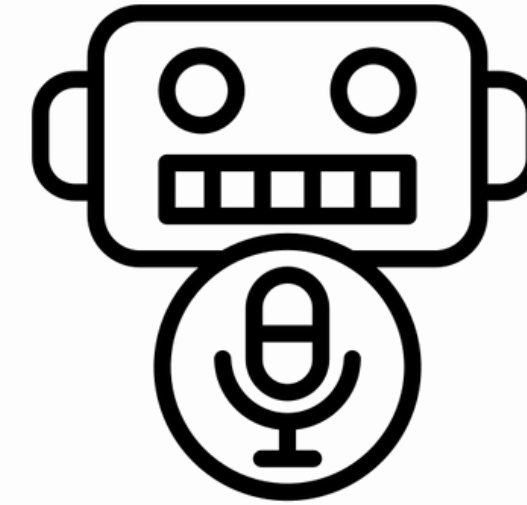
Adaptam o conteúdo e o ritmo às necessidades individuais, fornecendo feedback imediato (Escueta et al., 2017), (Bernacki; Greene; Crompton, 2020).



Disponibilidade contínua

Permitem o aprendizado informal por meio dos dispositivos móveis, sendo independente de tempo e espaço (Buchner; Andujar, 2019).

A metanálise de Lyu et al. (2025), quantificou os resultados de 31 pesquisas quase-experimentais comparando o aprendizado de uma segunda língua com e sem uso de chatbots.



MÓVEL

O aprendizado em dispositivos móveis ($g = 0,790$) demonstrou ser mais impactante do que em apenas computador ($g = 0,189$), o que se alinha à onipresença dos celulares (98.8% dos brasileiros) permitindo o aprendizado informal, independente de tempo e espaço.

IA GENERATIVA

A IA generativa ($g = 0,833$) mostrou um impacto superior no aprendizado em comparação com sistemas que não a utilizam ($g = 0,473$), o que justifica a adoção de LLMs para o aprendizado de idiomas.

SUPORTE A VOZ

O suporte a voz ($g = 0,809$) teve um impacto significativamente maior no aprendizado do que sistemas de apenas texto ($g = 0,425$).

Fator moderador	Condição	<i>g</i> de Hedges
Mobilidade	Dispositivo móvel	0,790***
	Apenas computador	0,189*
Modalidade	Suporte a voz	0,809***
	Apenas texto	0,425***
Tecnologia	IA generativa	0,833***
	Baseado em regras	0,473***

Nota. Valores de **g** (tamanho do efeito calculado) próximos a 0.5 indicam um efeito médio.

* $p < 0,05$. *** $p < 0,001$.

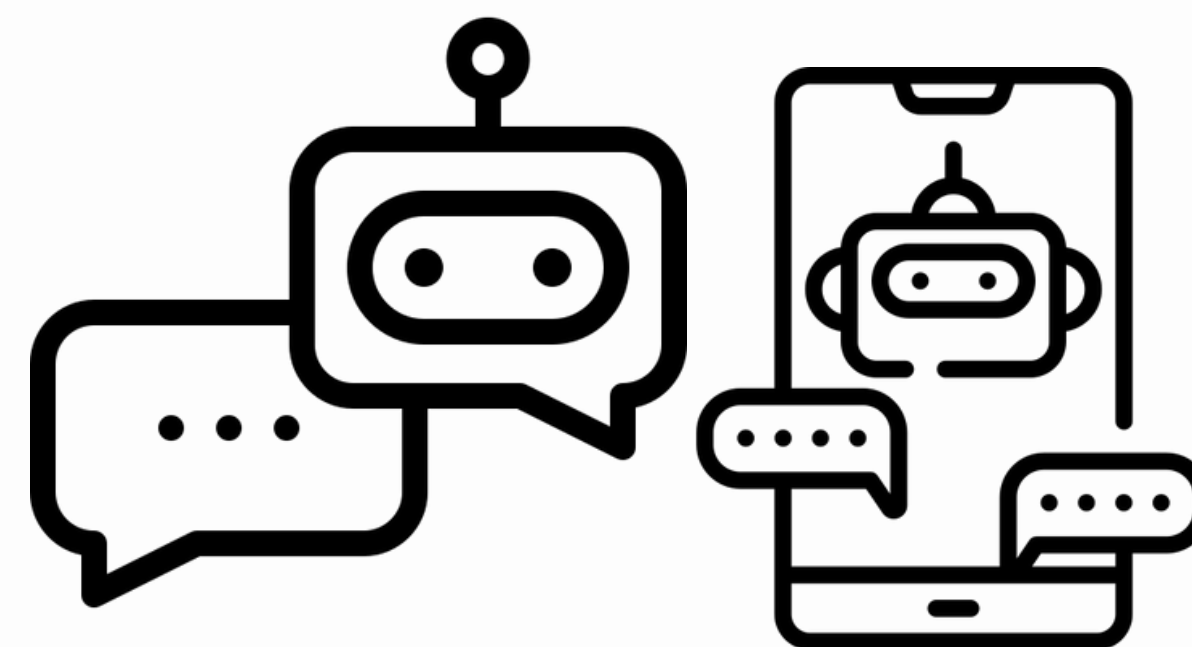
O projeto se posiciona exatamente na interseção de maior impacto pedagógico: IA generativa, por voz e por meio de um aplicativo para dispositivos móveis.

JUSTIFICATIVA

A implementação através de arquiteturas de agentes modernas na educação, como a proposta nesse trabalho, permanece pouco explorada, especialmente no contexto brasileiro.

LACUNA

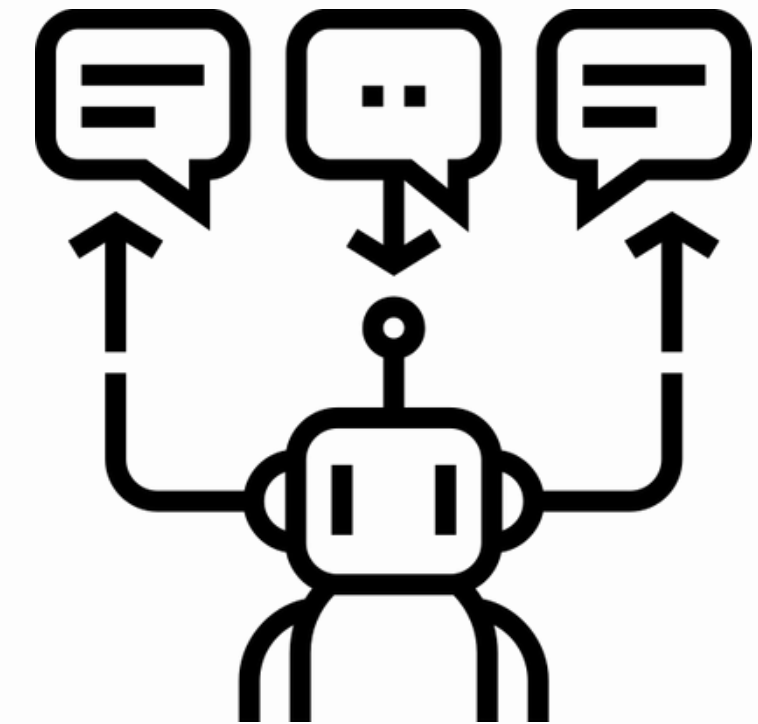
Este trabalho busca preencher esta lacuna, desenvolvendo um aplicativo por voz com IA que implemente uma arquitetura de agente moderna aplicada ao contexto educacional.



FUNDAMENTAÇÃO TEÓRICA

O DILEMA DOS TUTORES DE IA

O desenvolvimento de tutores de IA eficientes enfrenta um dilema entre a fluidez da comunicação e a precisão das informações fornecidas. Esse dilema tem como origem as abordagens de IA, neurais e simbólicas (Marcus, 2018), (Shahzad et al., 2025).



Puramente neurais (LLMs)

São modelos que podem alucinar, e no contexto educacional, frequentemente fornecem informações erradas ao aluno (Marcus, 2018)(Shahzad et al., 2025).

Puramente simbólicas (Regras)

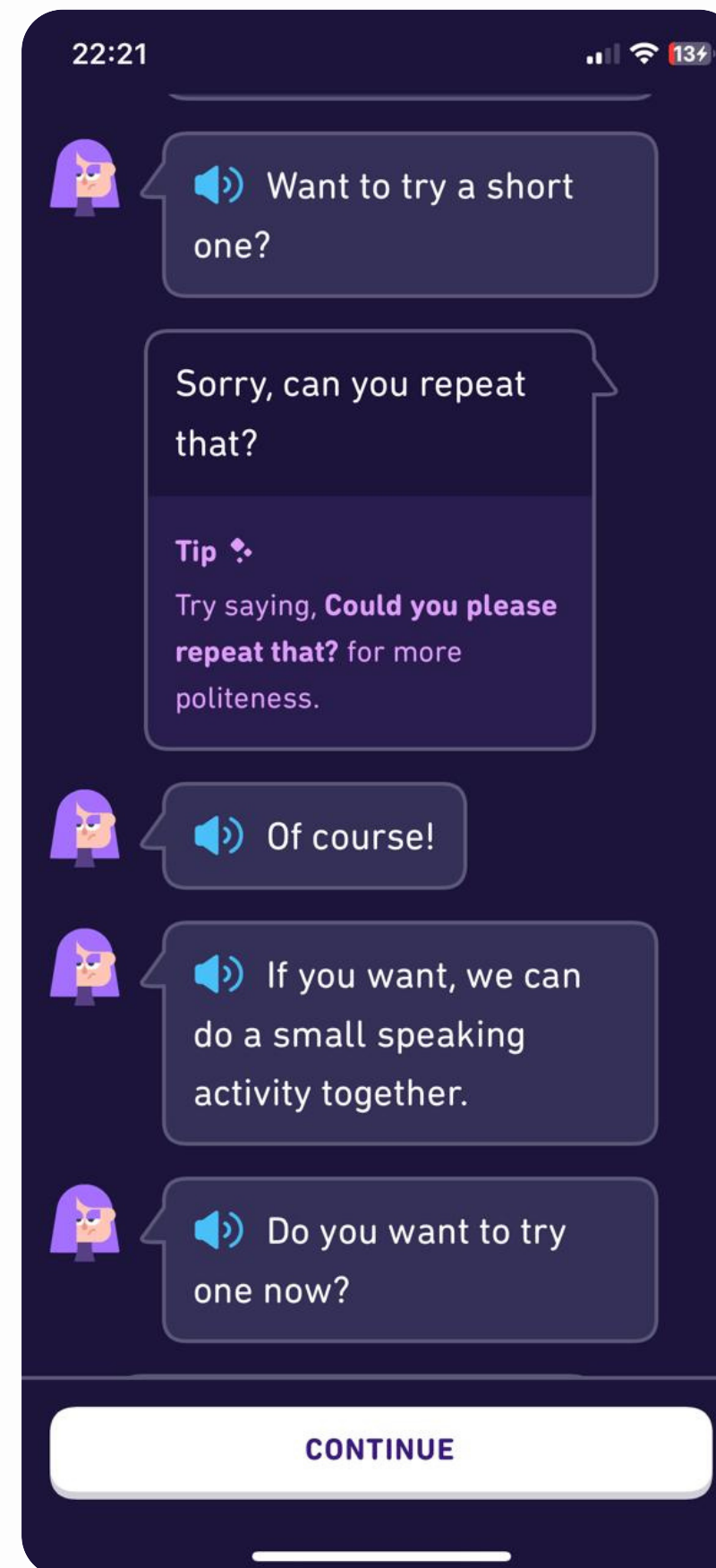
São determinísticas (precisas), mas excessivamente rígidas, por isso, Falham em capturar a fluidez da conversação natural (Mira, 2008).

Mistas (Neuro simbólicas)

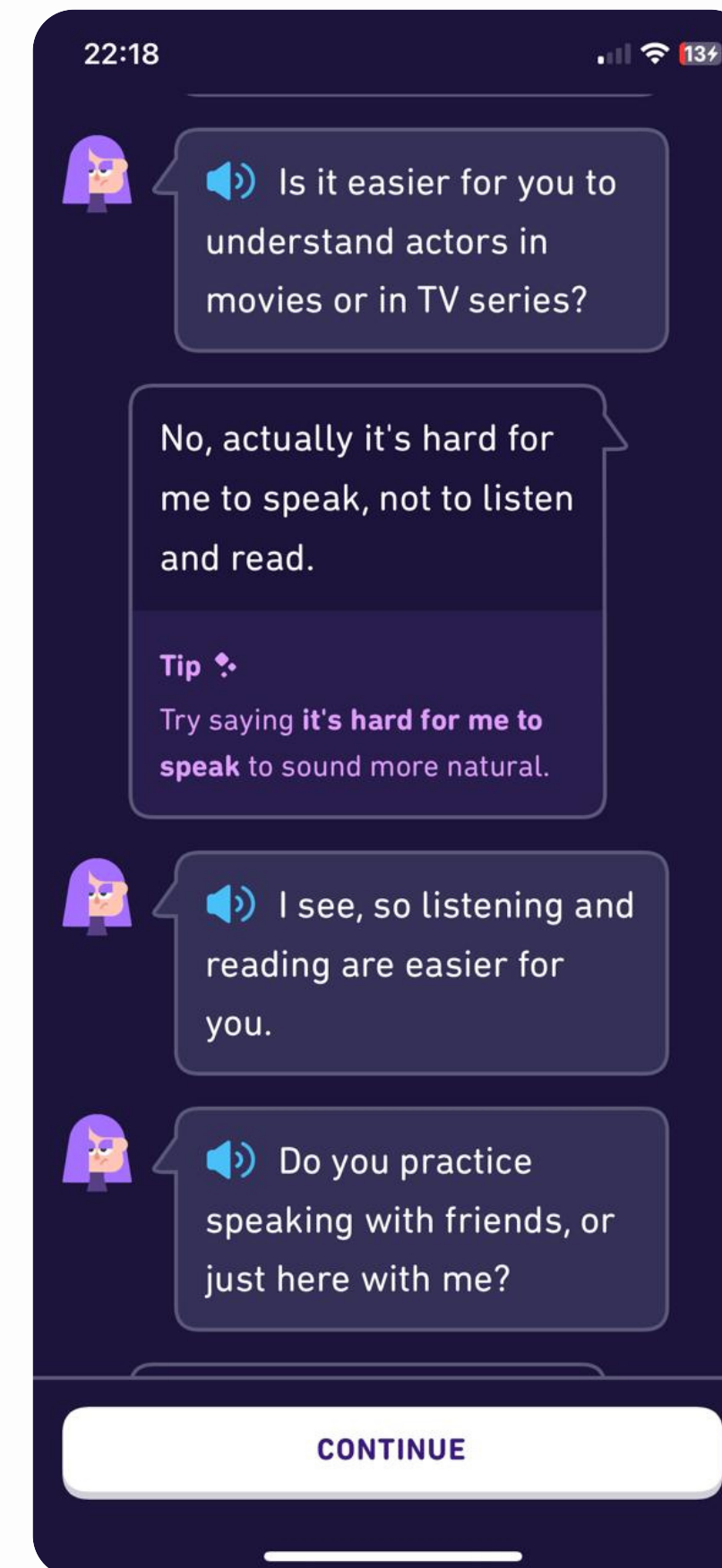
Buscam unir as abordagens neurais com as abordagens simbólicas. (Yu et al., 2023).

A figura (a) demonstra o uso de uma abordagem de agente único puramente neural, como a do Duolingo MAX. A figura (b) exibe um exemplo de imprecisão.

Fonte: Autoria própria.



(a)



(b)

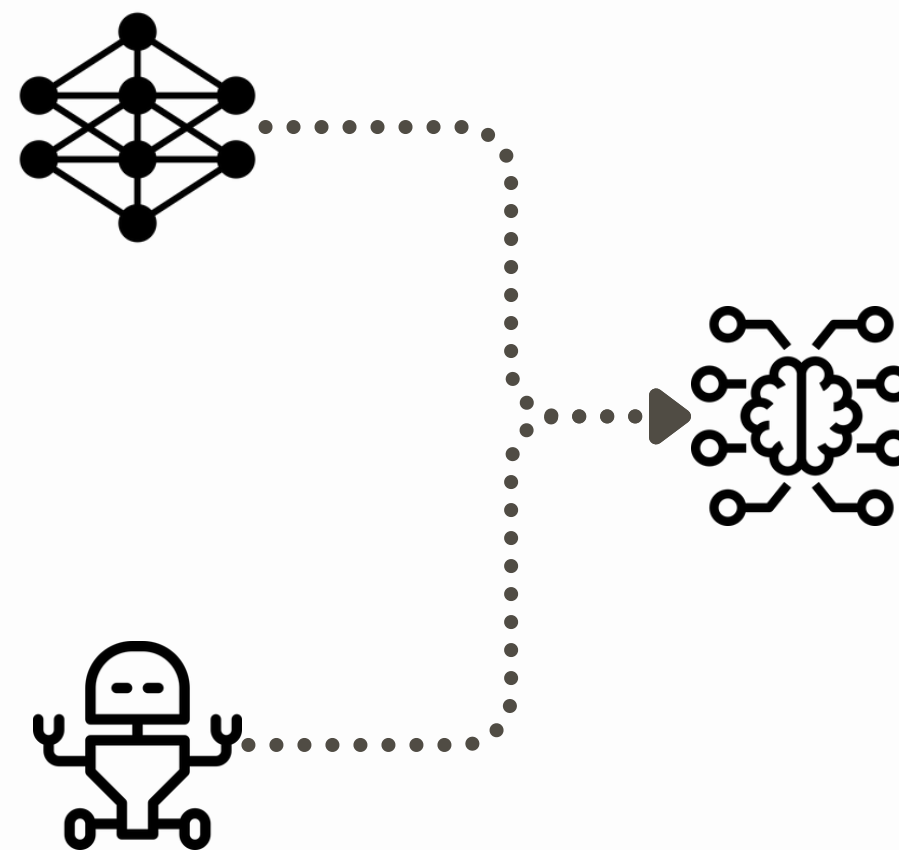
ABORDAGEM NEURO SIMBÓLICA

MODELOS NEURAI

Os modelos neurais possuem alta adaptabilidade e capacidade de reconhecer padrões, porém são conhecidos como caixa preta pois não é possível prever a origem do conhecimento, além de frequentemente apresentarem informações incorretas (Yu et al., 2023), (Caseli; Nunes, 2024).

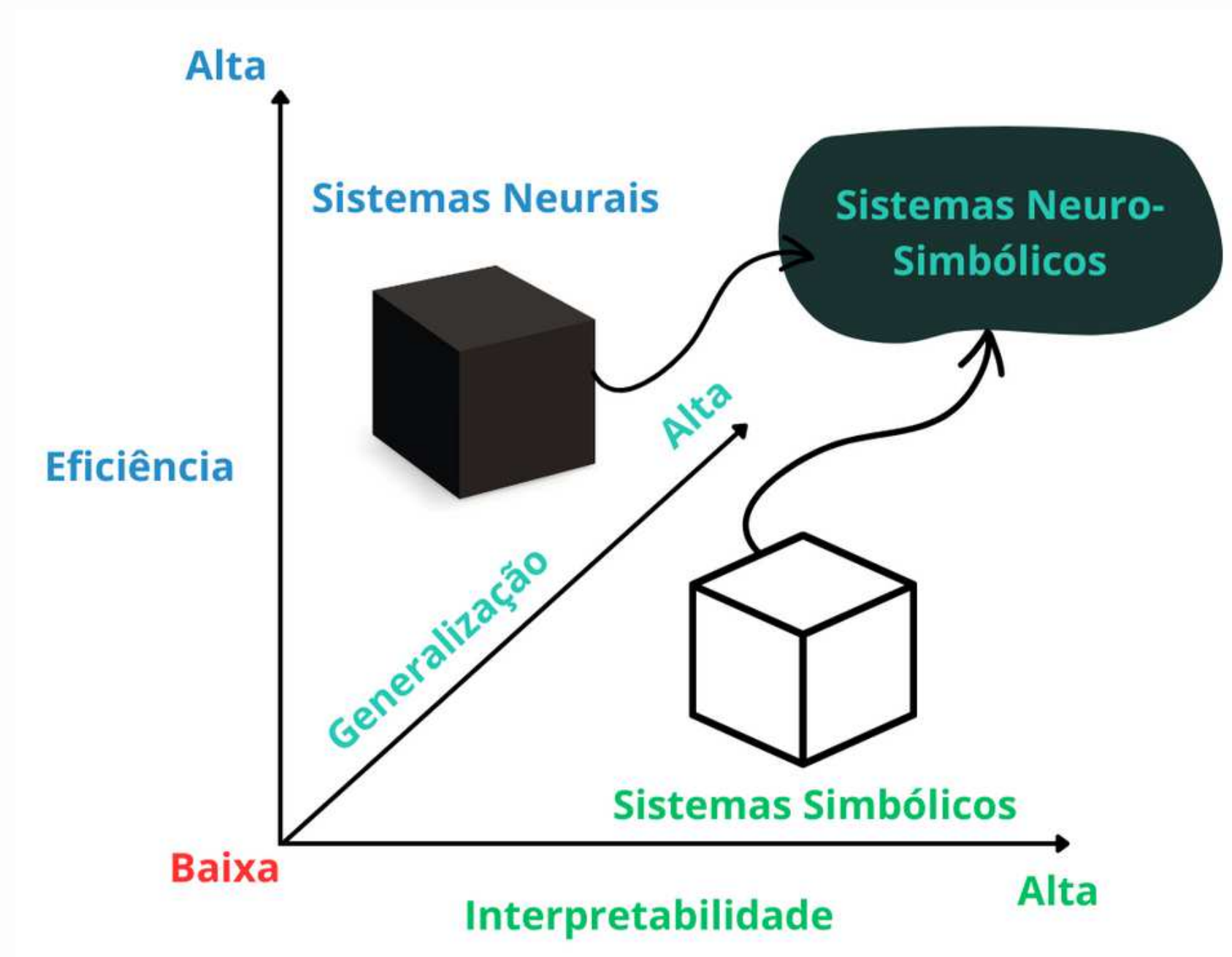
MODELOS SIMBÓLICOS

Os modelos simbólicos (conhecidos como sistemas especialistas) possuem raciocínio preciso e também verificável mas geram pouca profundidade em conversação natural.



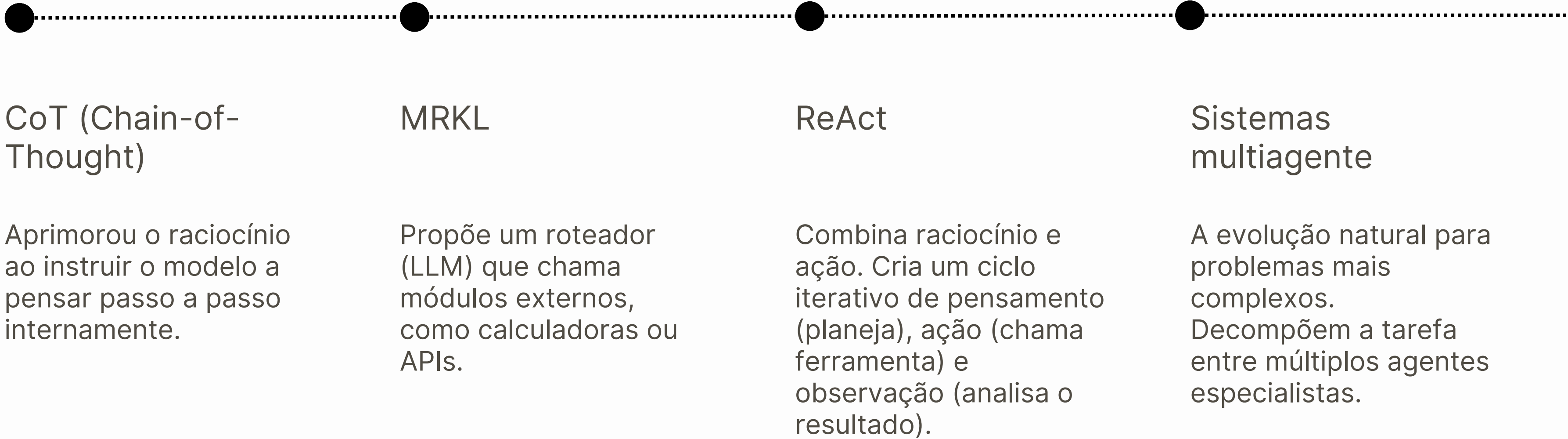
MODELOS NEURO SIMBÓLICOS

Busca unir o aprendizado estatístico dos sistemas neurais com o raciocínio estruturado dos sistemas simbólicos, visando potencializar a generalização. (Yu et al., 2023), (Caseli; Nunes, 2024).



Sistema neuro simbólico: A figura (adaptada de Yu et al., 2023), mostra como a abordagem neuro simbólica busca atingir simultaneamente alta eficiência, generalização e interpretabilidade.

A EVOLUÇÃO DOS PARAGIDMAS DE AGENTE



COT (CHAIN-OF-THOUGHT)

Se trata de uma técnica de prompting que demonstrou que, se pedirmos a um modelo de linguagem (LLM) para pensar passo a passo antes de dar a resposta final, ele melhora sua capacidade de raciocínio em tarefas complexas (Wei et al., 2022).

Como funciona

O modelo gera uma sequência de pensamentos internos que levam à solução.

O problema (limitação)

O CoT possui uma natureza interna. O modelo pensa muito bem, melhorando a precisão, mas não pode interagir com o mundo exterior durante esse processo. Não sendo possível incluir o raciocínio estruturado no processo.

Prompting padrão

Q: Rogério tem 5 bolas de tênis. Ele compra mais 2 latas de bolas de tênis. Cada lata possui 3 bolas de tênis. Quantas bolas de tênis ele possui agora?

A: A resposta é 11.

Q: A lanchonete tinha 23 maçãs. Se eles usaram 20 para fazer o almoço e compraram mais 6, quantas maçãs eles possuem agora?

Output

A: A resposta é 27. ✗

Chain of Thought Prompting

Q: Rogério tem 5 bolas de tênis. Ele compra mais 2 latas de bolas de tênis. Cada lata possui 3 bolas de tênis. Quantas bolas de tênis ele possui agora?

A: Rogério iniciou com 5 bolas. 2 latas de 3 bolas de tênis cada dá 6 bolas de tênis. $5 + 6 = 11$. A resposta é 11

Q: A lanchonete tinha 23 maçãs. Se eles usaram 20 para fazer o almoço e compraram mais 6, quantas maçãs eles possuem agora?

Output

A: A lanchonete tinha 23 maçãs originalmente. Eles usaram 20 para fazer o almoço. Então eles possuem $23 - 20 = 3$. Eles compraram 6 novas maçãs, logo eles possuem $3 + 6 = 9$. A resposta é 9 ✓

Prompt regular vs CoT (Adaptado de Wei et al.)

MRKL (MODULAR REASONING, KNOWLEDGE, AND LANGUAGE)

A MRKL é um padrão de arquitetura para construir sistemas inteligentes (agentes). A ideia central é que um LLM (que fornece o *Language* e *Reasoning*) não deve tentar fazer tudo sozinho, ele deve atuar como um roteador que consulta módulos externos especializados (que fornecem o *Knowledge*), (Karpas et al., 2022).

Como funciona

O sistema é composto por um roteador central (o LLM) e um conjunto de módulos (ferramentas, APIs, bancos de dados).

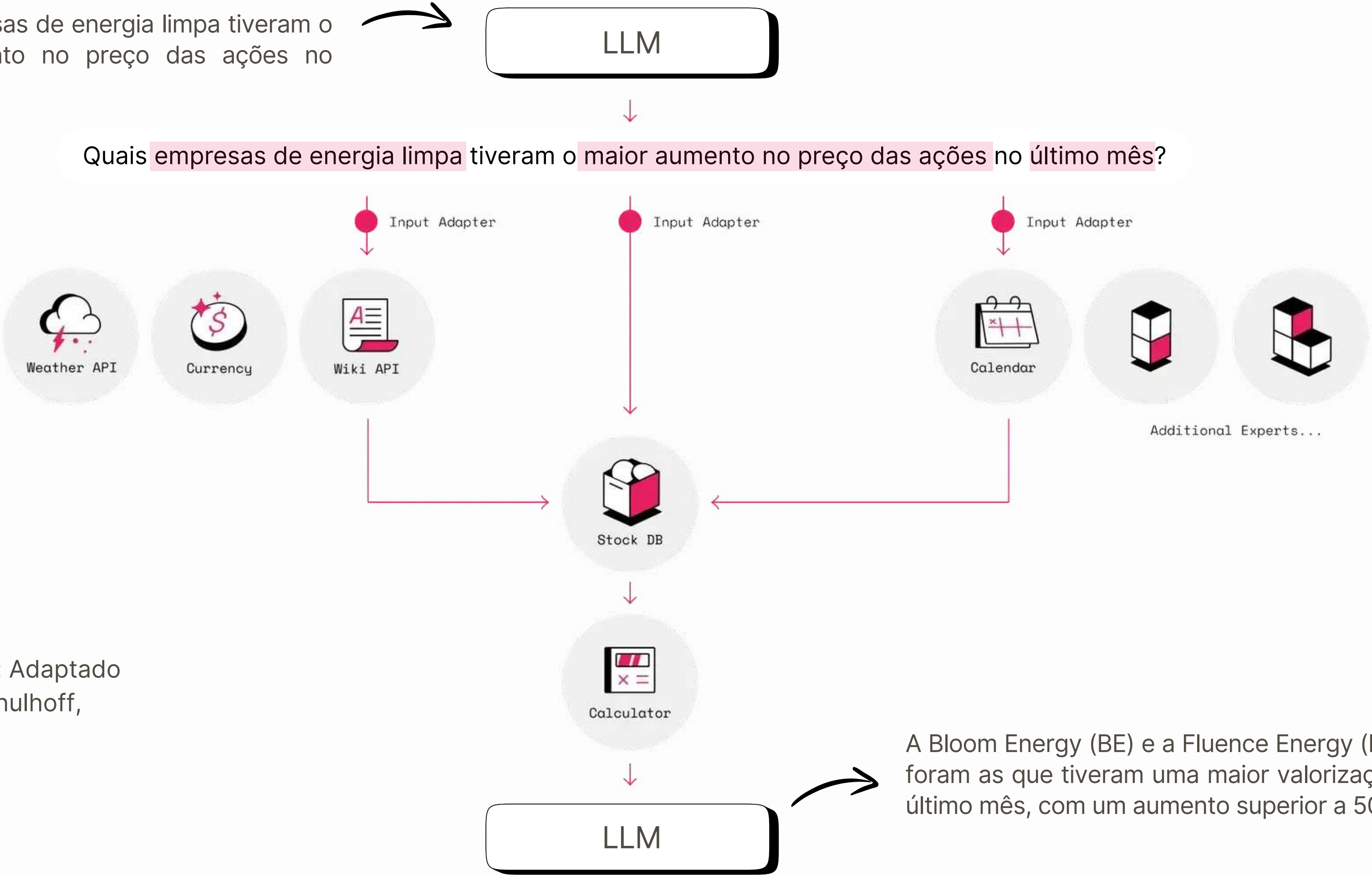
O problema (limitação)

A arquitetura MRKL define o que são os componentes (o LLM e os módulos), mas não especifica exatamente como o LLM deve coordenar esses módulos para resolver tarefas mais complexas. O problema surge quando um problema não pode ser resolvido com uma única chamada de ferramenta, exigindo uma sequência de etapas decompostas.



Fonte: Adaptado
de Karpas et al.,
2022.

Quais empresas de energia limpa tiveram o maior aumento no preço das ações no último mês?



Fonte: Adaptado de Schulhoff, 2023.

A Bloom Energy (BE) e a Fluence Energy (FLNC) foram as que tiveram uma maior valorização no último mês, com um aumento superior a 50%.

REACT (REASONING AND ACTING)

O ReAct combina o raciocínio (inspirado no CoT) com a capacidade de agir (usar ferramentas do MRKL) operando em um ciclo de etapas (Yao et al., 2023).

Como funciona

1. **Thought** (pensamento): O agente analisa o estado atual e a tarefa. Ele usa um processo similar ao CoT para decidir qual é o próximo passo ou qual informação está faltando.
2. **Act** (ação): Com base no pensamento, o agente decide executar uma ação. Essa ação é geralmente o uso de uma ferramenta externa.
3. **Observation** (observação): O agente recebe o resultado (a observação) da ferramenta.

Esse resultado alimenta a próxima etapa de thought, e o ciclo se repete até que a tarefa seja concluída.

O problema (limitação)

Quando uma tarefa é complexa, um único agente ReAct começa a falhar e essas falhas ocorrem muito por conta da sua janela de contexto limitada, esquecendo o objetivo inicial. Além de muitas vezes propagar o erro que cometeu (viés de confirmação).

$$(\tau_1, a_1, o_1, \tau_2, a_2, o_2, \dots, \tau_T)$$

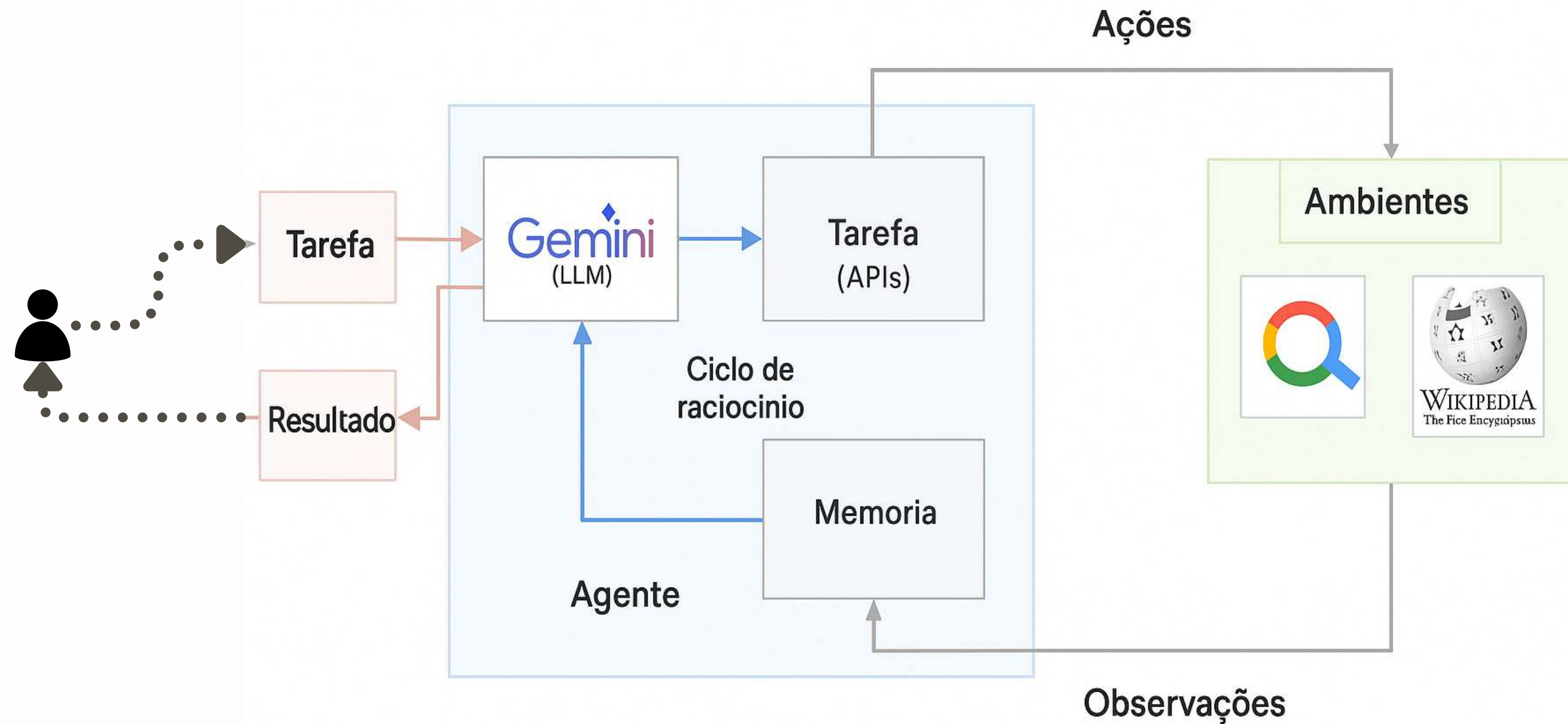
trajetória linear do agente ReAct

τ é o pensamento usado durante o planejamento

a é a ação na qual o agente decide usar uma ferramenta

o é a observação, sendo o resultado que a ferramenta retorna, que o agente processa

(LI et al., 2025)



Fonte: Adaptado de (Shankar. 2024).

ARQUITETURAS MULTIAGENTE

A arquitetura multiagente surge como uma solução para o problema da ReAct. Ao invés de um único agente generalista tentando resolver uma tarefa complexa de forma completa, cria-se múltiplos agentes de IA que colaboram para atingir o objetivo (Hadfield; Barry; Zhang, 2024), (Li et al., 2025).

Como funciona

Em vez de um agente generalista, criamos uma equipe de agentes especialistas. (dividir para conquistar).

O problema (limitação)

O problema principal passa a ser lidar com o alto custo, já que cada mensagem ou pensamento de cada agente é uma chamada ao LLM. Consequentemente, a alta latência impacta diretamente na experiência do usuário, obrigando-o a aguardar muito tempo por uma resposta.

$$s_j^t = f_j(s_j^{t-1}, \{m_{i \rightarrow j}^{t-1}\}_{a_i \in \mathcal{A}})$$

função de transição de estado

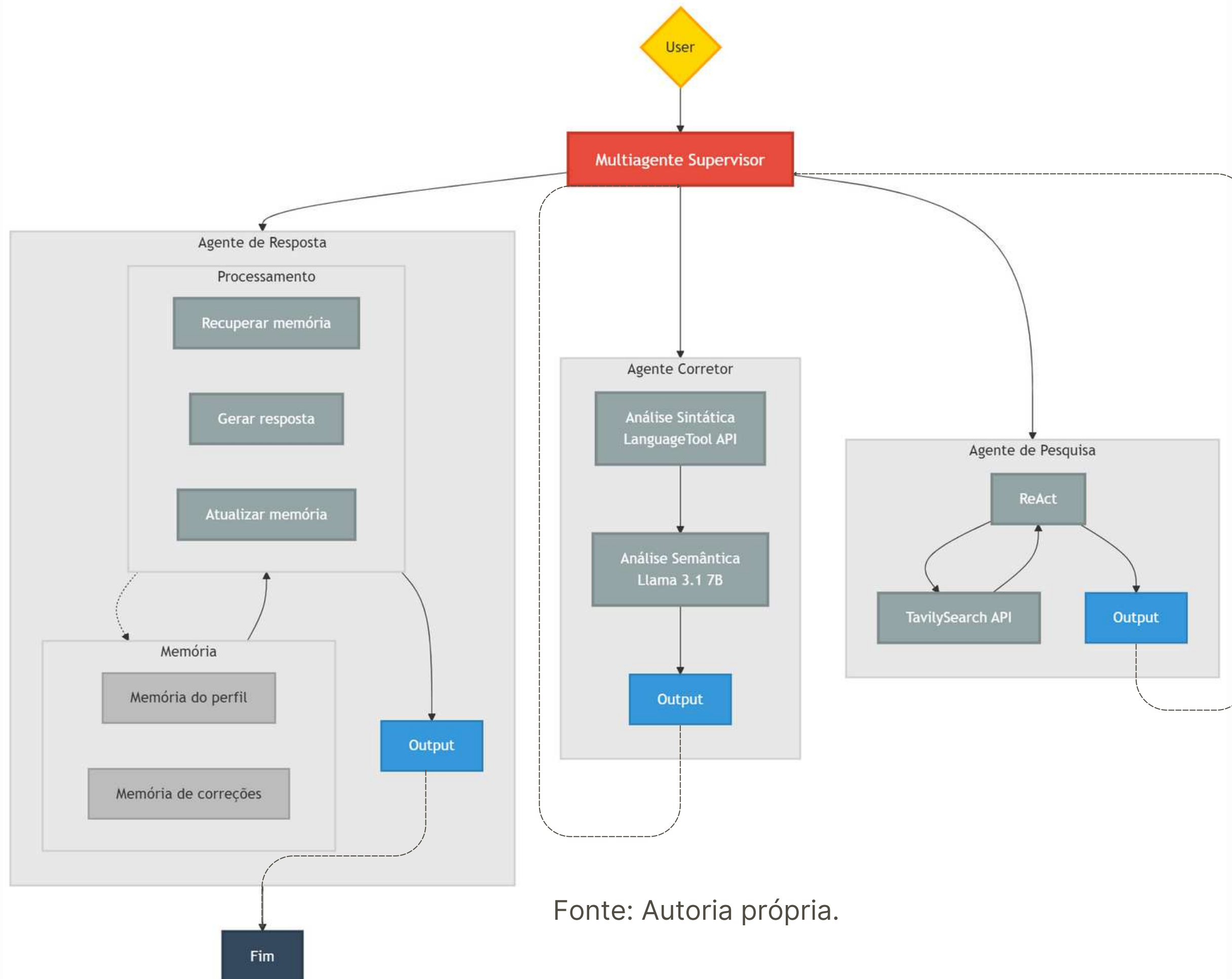
s_j^t O estado do agente j no tempo t (o que o agente j sabe)

f_j função interna do agente j (sua lógica)

s_j^{t-1} estado anterior do agente j

$\{m_{i \rightarrow j}^{t-1}\}$ conjunto de mensagens enviadas do agente i para o agente j, para cada agente i (a_i) que exista no sistema $\mathcal{A}$

(Li et al., 2025)



Fonte: Autoria própria.

PROPOSTA E OBJETIVOS

O PARADIGMA CHAIN-OF-AGENTS (COA)

O Chain-of-Agents (CoA) é um novo paradigma de raciocínio de LLM (Li et al, 2025), que treina um único modelo para resolver problemas complexos, simulando a colaboração que normalmente exigiria um sistema multiagente (com múltiplos agentes e ferramentas).

O resultado

O resultado desses modelos treinados com CoA é chamado de Agent Foundation Models (AFMs).

O benefício

A comunicação e colaboração entre agentes ocorre de forma interna, em um único modelo, com isso, o CoA resolve a alta latência e altos custos dos sistemas multiagente.

O resultado citado no paper é a redução de **84.6% menos tokens** em comparação com os sistemas multiagente tradicionais. A abordagem do CoA também permite que o modelo melhore seu desempenho ao ser treinado com mais dados.

AGENT FOUNDATION MODEL (AFM)

Geração de dados (destilação)

Primeiro, é usado um sistema multiagente tradicional (MAS) para resolver diversas tarefas complexas.

Destilação multiagente

Em seguida, é coletado todo o processo de colaboração, comunicação e uso de ferramentas desse sistema multiagente e o destilam em uma única trajetória de texto.

Fine Tuning (SFT e RL)

Um modelo de linguagem (LLM) base é então treinado (via *Supervised Fine-Tuning* e *Agentic Reinforcement Learning*) nessas trajetórias de CoA geradas anteriormente.

Resultado (inferência)

O modelo final (o AFM) aprende a desempenhar todos os papéis (planejador, executor, crítico) sozinho. Ele ativa dinamicamente diferentes agentes de ferramenta e agentes de papéis (como Plan Agent, Reflection Agent, Search Agent) dentro de seu próprio processo de geração, em vez de chamar outros modelos.

$$\mathcal{S}_t = f_{\theta}(\mathcal{S}_{t-1}, \phi_{t-1}, o_{t-1})$$

função de transição de estado

$\mathcal{S}_t$ Estado atual de raciocínio (tudo o que o agente sabe no tempo t).

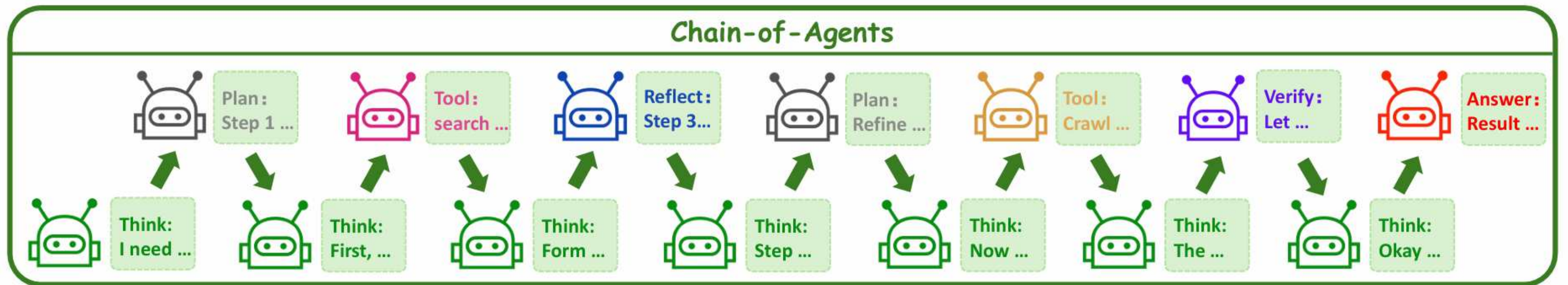
f_{θ} Função do próprio AFM no qual Θ representa todos os parâmetros que o modelo aprendeu durante o treinamento (destilação, SFT e RL)

$\mathcal{S}_{t-1}$ Estado anterior (o que o agente sabia no passo imediatamente anterior ao atual)

ϕ_{t-1} o papel (agente) que foi ativado no passo anterior, podendo ser por exemplo um agente de planejamento ou busca

o_{t-1} O resultado (observação) da ação do papel ϕ_{t-1} (se fosse o papel de um agente de busca, o resultado seria os resultados da busca)

(LI et al., 2025)



Fonte: (Li et al, 2025).

O novo estado de conhecimento do agente $\mathcal{S}_t$ é determinado pelo próprio modelo AFM f_θ ao processar seu estado de conhecimento antigo $\mathcal{S}_{t-1}$, o papel que ele acabou de desempenhar ϕ_{t-1} e o que ele aprendeu com isso $\mathcal{O}_{t-1}$.

Com base em tudo o que ele sabe nesse momento, o modelo AFM decide o que fazer, se deve ativar o agente de plano ϕ_{plan} , se deve ativar o agente de busca ϕ_{search} , se deve ativar o agente de reflexão $\phi_{reflection}$, etc.

OBJETIVOS DO TRABALHO

Objetivo geral

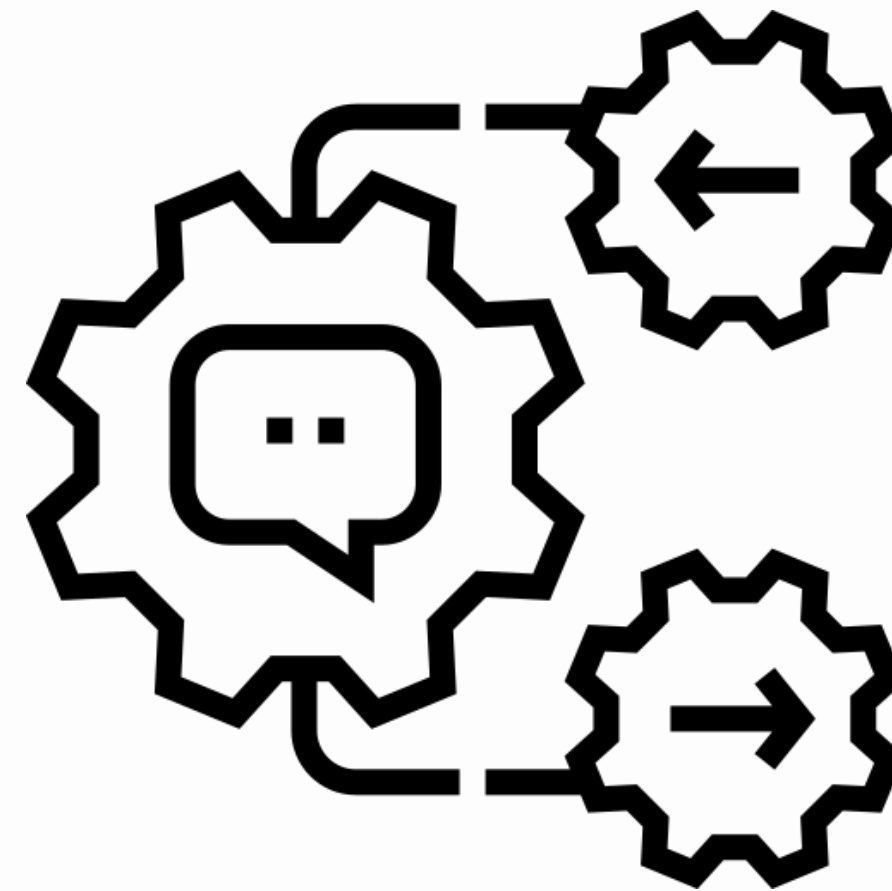
- Desenvolver e validar um tutor de IA conversacional (via aplicativo móvel), utilizando uma arquitetura composta com base no paradigma CoA, a fim de proporcionar uma experiência fluida e pedagogicamente eficaz.

Objetivos específicos

- **Pipeline de análise gramatical:** Implementar um pipeline de processamento em camadas (usando LangGraph) para análise linguística, integrando conhecimento estruturado (verificação sintática) e neural (análise semântica e contextual).
- **Agente central:** Desenvolver um agente central unificado capaz de receber as análises do pipeline, gerar planos de ação dinâmicos, invocar ferramentas e realizar autocrítica.

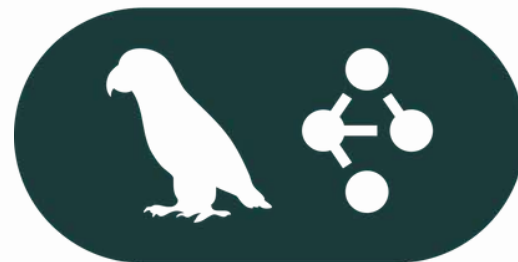
MATERIAIS E MÉTODOS

IMPLEMENTAÇÃO DO SERVIDOR



A ARQUITETURA DO AGENTE

O sistema utiliza uma arquitetura que separa a correção gramatical (para maior precisão) do raciocínio do agente central conversacional (para maior fluidez). Isso foi implementado em dois componentes principais.



Pipeline de análise gramatical

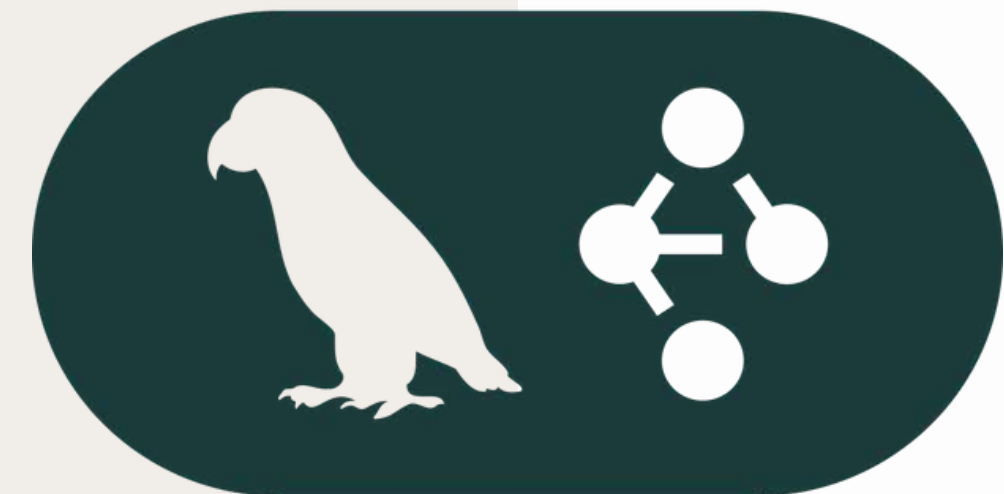
Um pipeline de correção em LangGraph focado exclusivamente em analisar a entrada do usuário.

Componente principal de raciocínio

O agente CoA que recebe a análise do pipeline e o histórico para formular a resposta.

LANGGRAPH

O LangGraph é u.m framework de agentes de IA que implementa interações de agente por meio de um grafo. Em vez de encadear chamadas de modelo de forma linear, a lógica do agente é feita um conjunto de nós (cada nó sendo uma etapa como invocar um LLM ou executar uma função) conectados por arestas que definem o fluxo de controle (qual nó será executado em seguida). Em vez de cada etapa ser independente, o LangGraph mantém um estado central compartilhado que é lido e modificado pelos nós, permitindo que o agente lembre o que aconteceu em etapas anteriores (Pelluru, 2025).



O PIPELINE DE ANÁLISE GRAMATICAL

Foi Implementado utilizando LangGraph com o objetivo de processar a entrada do usuário em camadas de correções (sintática, semântica e contextual).

Objetivo:

Garantir a precisão na correção da mensagem do usuário.

Como funciona

1. **Nó de análise sintática (simbólico):** A entrada do usuário é enviada primeiramente para a Language Tool API que realiza uma análise sintática baseada em regras, identificando erros gramaticais sintáticos.
2. **Nó de análise semântica e contextual (neural):** Recebe a entrada original + a análise sintática, enviando para o Llama 3.1 70B com um prompt especializado para realizar uma análise semântica, detectando erros contextuais e de naturalidade (que soam estranhos para um nativo).

A saída deste pipeline é um objeto de correção estruturado, que é então enviado ao agente principal.

O AGENTE CENTRAL UNIFICADO

Implementado como um agente central que recebe a correção gramatical e opera como o agente central unificado através de engenharia de prompt.

Como funciona

O LLM (Llama 3.1 70B) é instruído a se comportar como um AFM, simulando internamente os diferentes papéis (planejador, crítico).

O ciclo de raciocínio

O agente é forçado a estruturar sua resposta usando tags específicas:

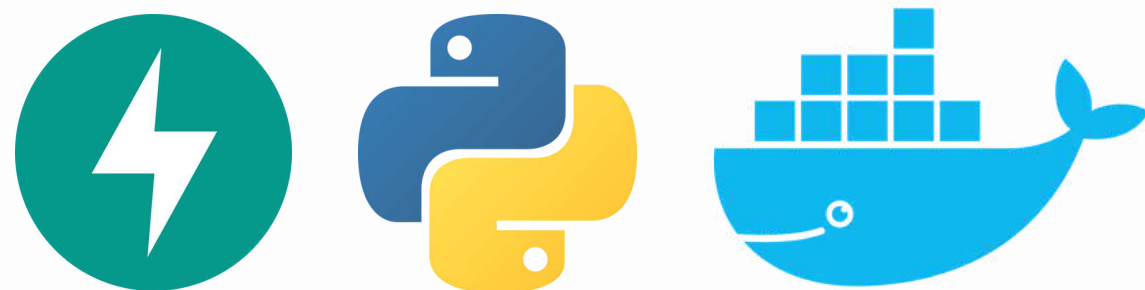
1. **(*<plan>*)** O agente deve gerar um plano de ação explícito antes de agir.
2. **(*<reflection>*)** Após uma ação, o agente reflete sobre o resultado, permitindo autocrítica e ajuste dinâmico do plano.
3. **(*<tool_call>*)** Permite que o agente venha a invocar ferramentas externas (WebSearch, UpdateUserProfile) para interagir com o mundo externo e o banco de dados.

ARQUITETURA DE SOFTWARE

Implementado por meio de uma arquitetura em camadas utilizando FastAPI (Python) com Docker seguindo *Clean Architecture* (Martin, 2017) a fim de promover clara separação de responsabilidades.

Camadas

1. **Entidades (SQLAlchemy)** O núcleo do domínio de negócio (camada mais interna).
2. **Serviços** Contêm a lógica de negócio; orquestram as entidades e os componentes do agente.
3. **Modelos de dados (Pydantic)** DTOs que definem os contratos da API (validação, serialização).
4. **Controladores:** A camada mais externa; gerencia os endpoints da API (rotas HTTP).



FASTAPI

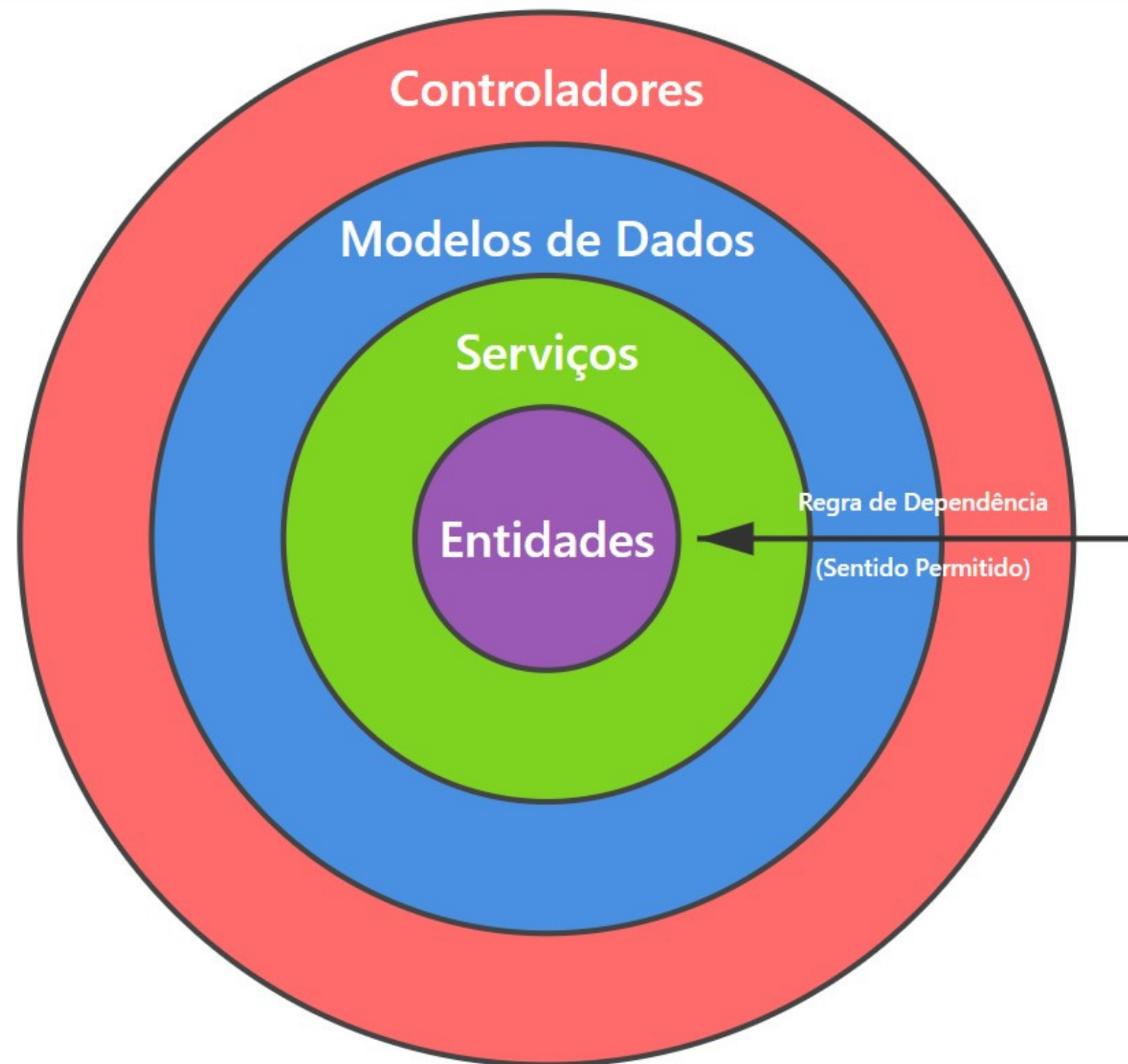
FastAPI é um framework Python moderno, projetado para simplicidade, velocidade e eficiência (Lubanovic, 2023).

Suporte assíncrono: Permite operações não bloqueantes, lidando com muitas solicitações.

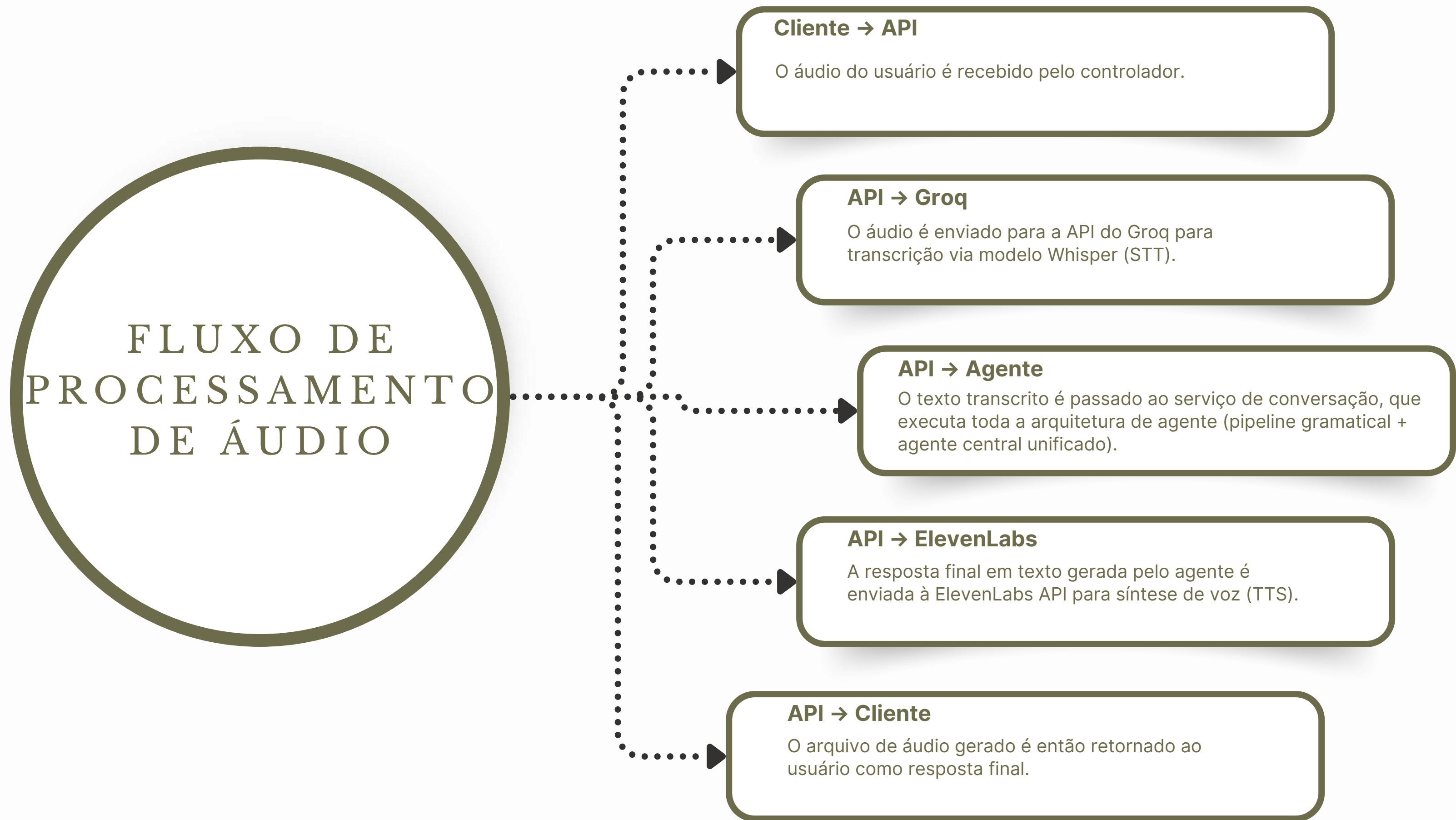
Validação de requisição: Usa modelos Pydantic para validar automaticamente os dados da solicitação.

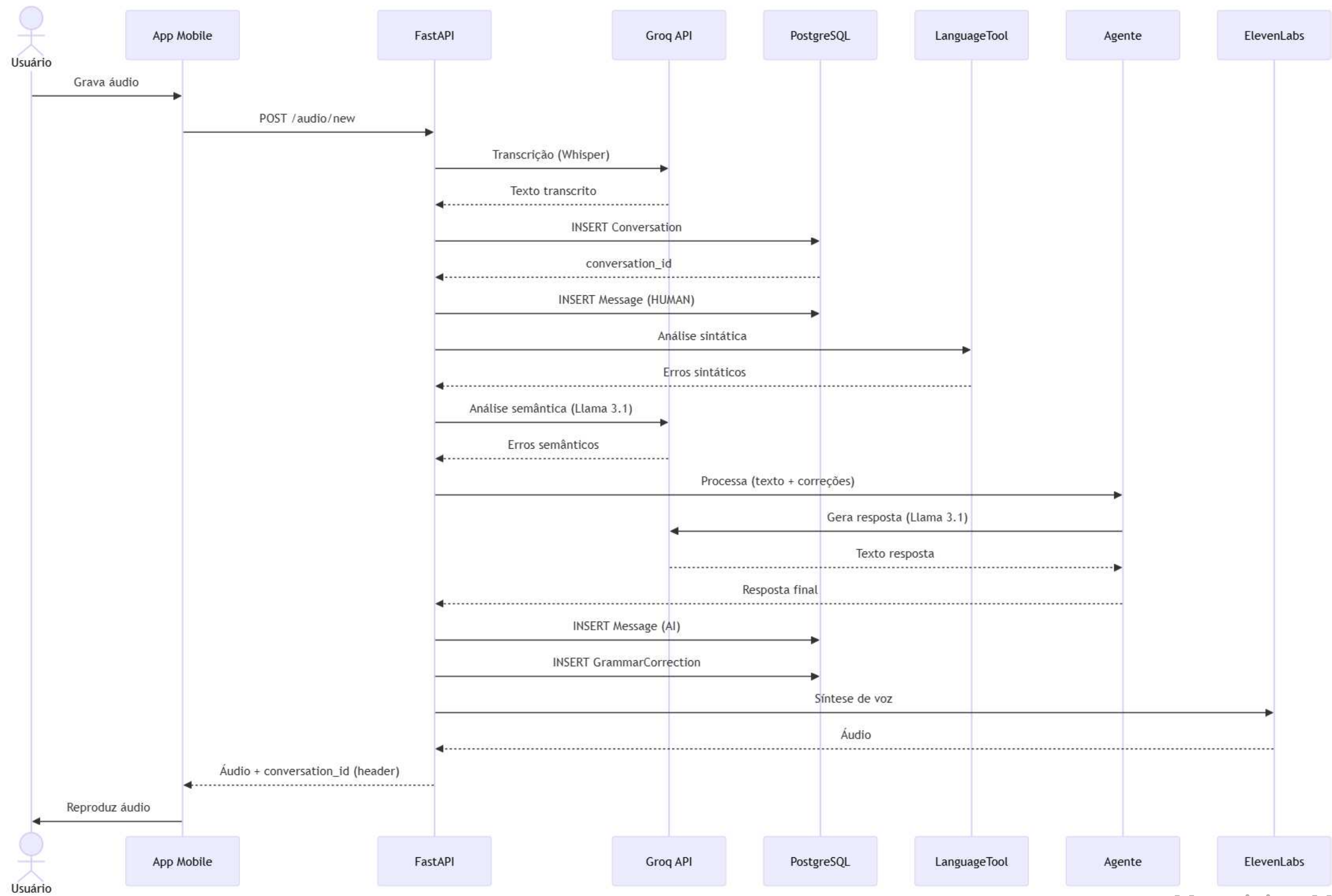
Documentação automática: Gera automaticamente documentos da API usando o padrão OpenAPI e Swagger a fim de facilitar os testes.



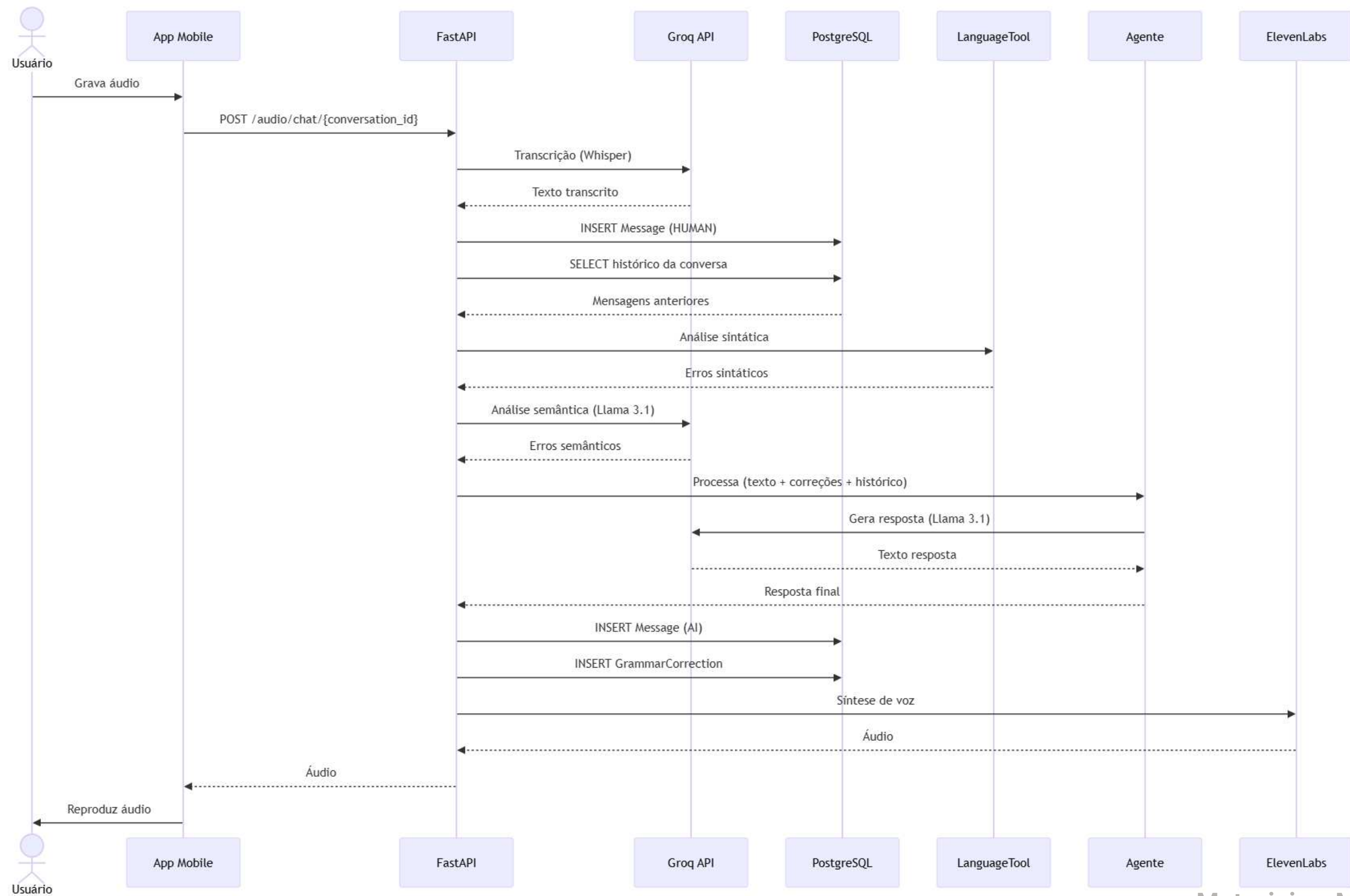


Fonte: Adaptado de Valente (2020).





Fonte: Autoria própria.



Fonte: Autoria própria.

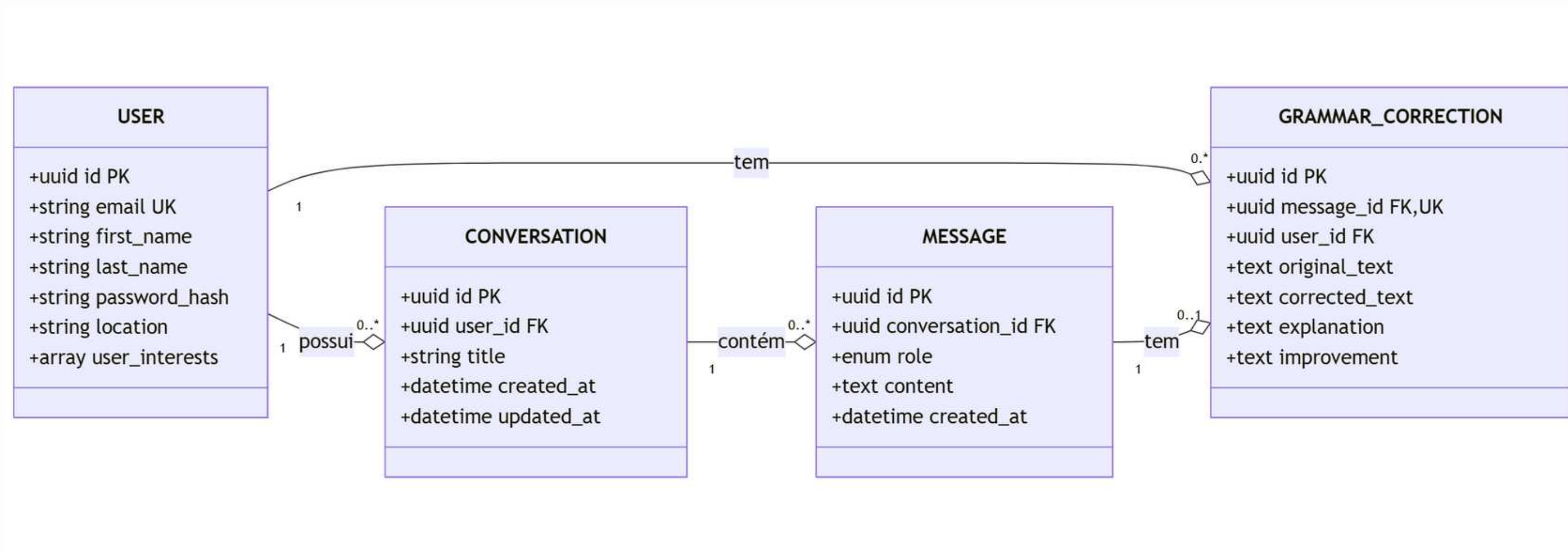
MODELAGEM DE DADOS

O banco de dados utilizado foi o PostgreSQL para armazenamento permanente com o SQLAlchemy para abstração de consultas SQL em Python.



Entidades

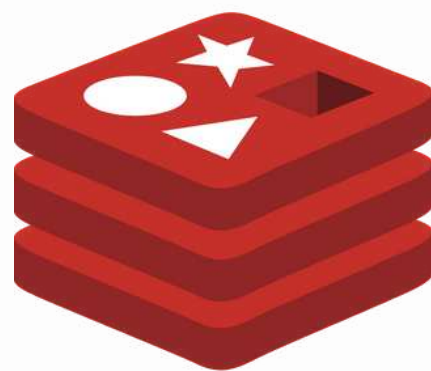
1. **User** Armazena dados de perfil e autenticação .
2. **Conversation** Representa uma sessão de conversa.
3. **Message** Uma única mensagem (HUMAN ou AI) dentro de uma *Conversation*.
4. **GrammarCorrection**: Armazena o resultado da análise gramatical, com relação 1:1 com uma Message.



Fonte: Autoria própria.

ESTRATÉGIA DE CACHE

Implementado com o objetivo de otimizar o tempo de resposta e reduzir a carga no banco de dados, sendo essencial para a personalização em tempo real dos interesses do usuário.



Padrão *Write-Through Cache* (cache com escrita direta)

1. **Escrita** Toda operação de escrita (atualizar perfil do usuário) é realizada simultaneamente no cache (Redis) e no banco (PostgreSQL).
2. **Leitura** O agente sempre lê primeiro o perfil do usuário do Redis (*Cache Hit*), o que garante acesso instantâneo.

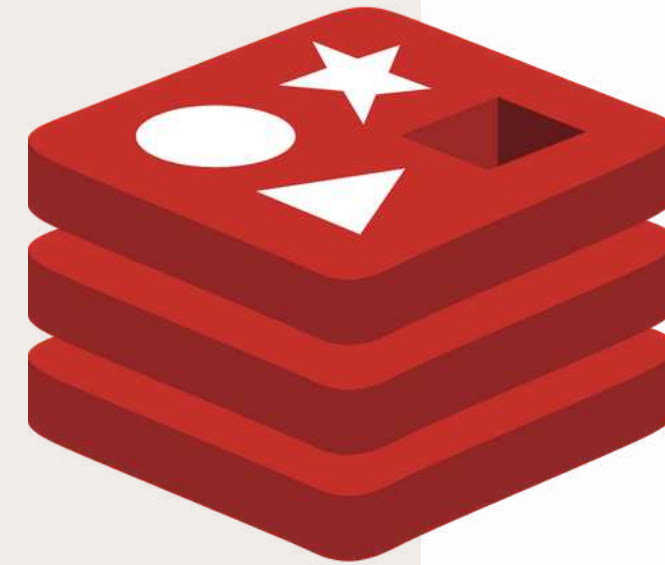
Benefício

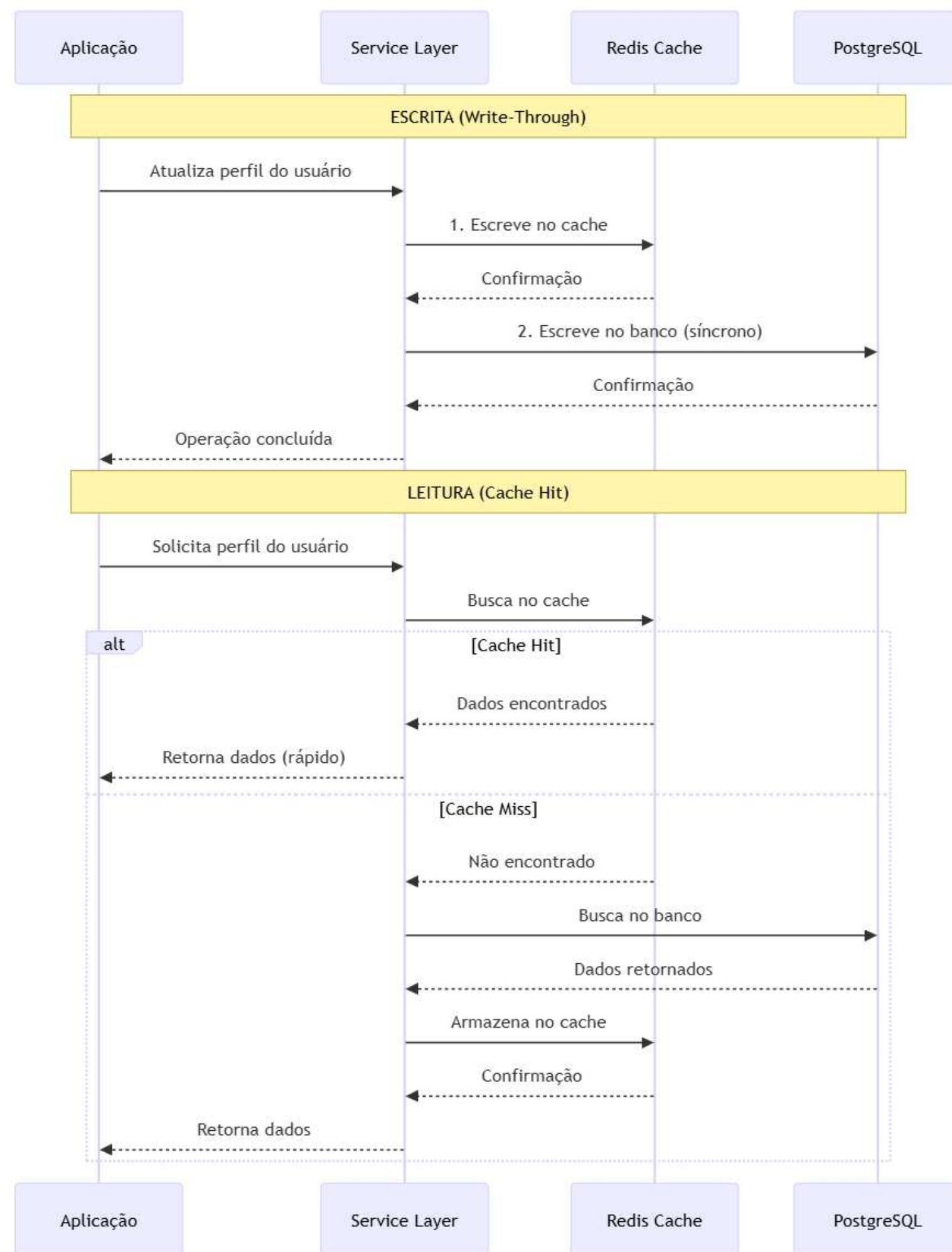
Garante tanto a consistência total dos dados (o cache nunca está desatualizado) quanto a performance de leitura (Jouppi, 1993).

REDIS

Redis é um banco de dados NoSQL de código aberto, que armazena dados no formato chave valor, possui foco em performance e é usado popularmente como cache (Carlson, 2013).

Armazenamento em memória: Guarda os dados primariamente na memória RAM, garantindo latência de microssegundos para leitura e escrita.





Fonte: Autoria própria.

Rotas

auth



POST /auth/ Register User



POST /auth/token Login For Access Token



Users



GET /users/me Get Current User



GET /users/me/profile Get Current User Profile



PUT /users/change-password Change Password



Conversations



GET /conversations/ List User Conversations



GET /conversations/{conversation_id} Get Conversation Details



Audio



POST /audio/new Create New Audio Conversation

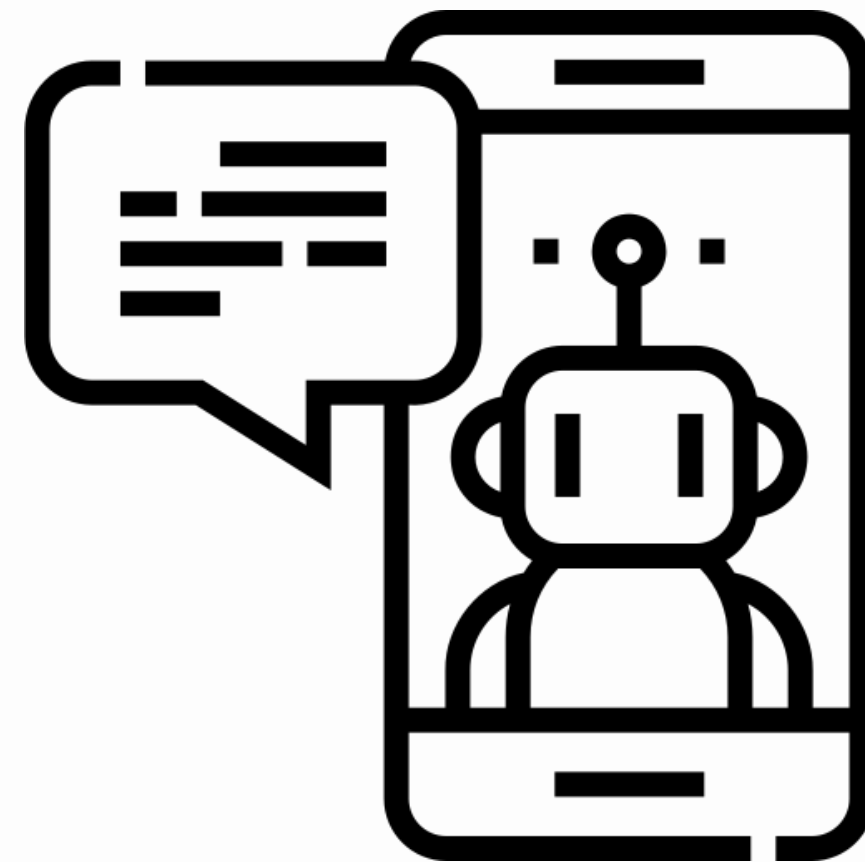


POST /audio/chat/{conversation_id} Continue Audio Conversation



Fonte: Autoria própria.

IMPLEMENTAÇÃO CLIENTE (MOBILE)



ARQUITETURA DA APLICAÇÃO MÓVEL

Implementado utilizando arquitetura MVVM ((Model-View-ViewModel)) com React Native e Expo (Typescript) a fim de promover clara separação de responsabilidades (Fuksa; Speth; Becker, 2024).

Camadas

1. **View** Componentes responsáveis apenas por renderizar a UI (Interface do usuário) e capturar interações do usuário.
2. **ViewModel** Atuam como ponte. Possuem toda a lógica de apresentação e gerenciamento de estado da UI.
3. **Model** A camada de dados e lógica de negócio, a exemplo os serviços de comunicação com a API.



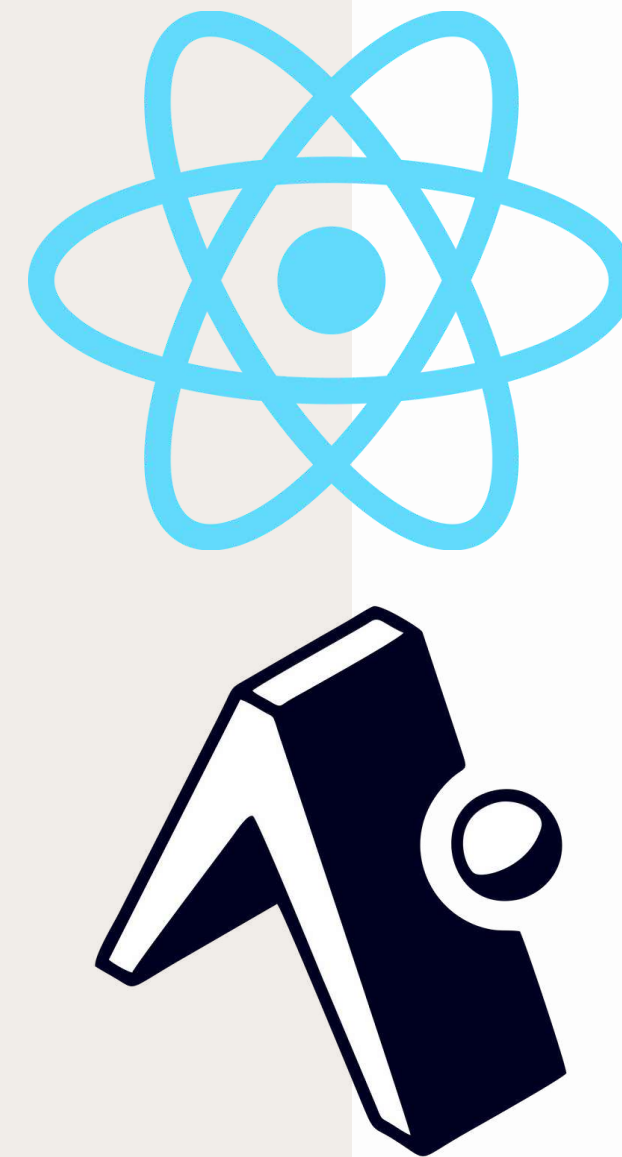
REACT NATIVE + EXPO

React Native é um framework para criar apps nativos com React, e Expo é uma plataforma que simplifica esse processo (Expo, 2025).

Multiplataforma: Permite compilar para aplicativos iOS e Android nativos.

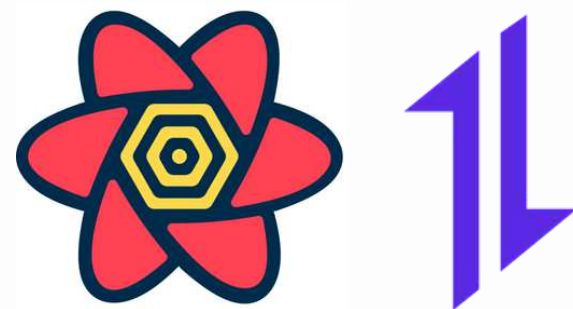
Desenvolvimento rápido (Expo): O Expo gerencia a complexidade do *build* nativo e oferece o app chamado Expo Go para visualizar as alterações no celular instantaneamente.

APIs unificadas (Expo): Fornece um conjunto de APIs para acessar facilmente os recursos do dispositivo como câmera, notificações, localização, etc, sem precisar escrever código nativo de cada sistema operacional.



GERENCIAMENTO DE ESTADO

Para o gerenciamento do estado dos dados recebidos pelo servidor foi utilizado as bibliotecas TanStack Query e Axios.



Soluções

1. **TanStack Query** Biblioteca usada para tratar automaticamente os estados de loading, error e success, além de gerenciar o cache no lado do cliente.
2. **Axios** Foi usado para anexar automaticamente o token de autenticação em todas as chamadas protegidas.
 - a. Se o token expirar ou o usuário tentar acessar uma rota protegida sem token, ele automaticamente é redirecionado para a tela de login.

PERSISTÊNCIA DE ÁUDIO

Permite que o usuário possa reouvir a última resposta de áudio do tutor em qualquer conversa, mesmo após fechar e reabrir o aplicativo.

Mecanismo

1. **Mapeamento** Mantém um mapa que associa o ID da conversa ao identificador do último arquivo de áudio que foi salvo no dispositivo.
2. **Substituição** Ao receber um novo áudio para uma conversa, o serviço apaga o arquivo de áudio anterior daquela conversa para economizar espaço.
3. **Limpeza** Ao carregar a lista de conversas, uma rotina é executada para remover arquivos de áudio que pertenciam a conversas que já foram deletadas pelo usuário.

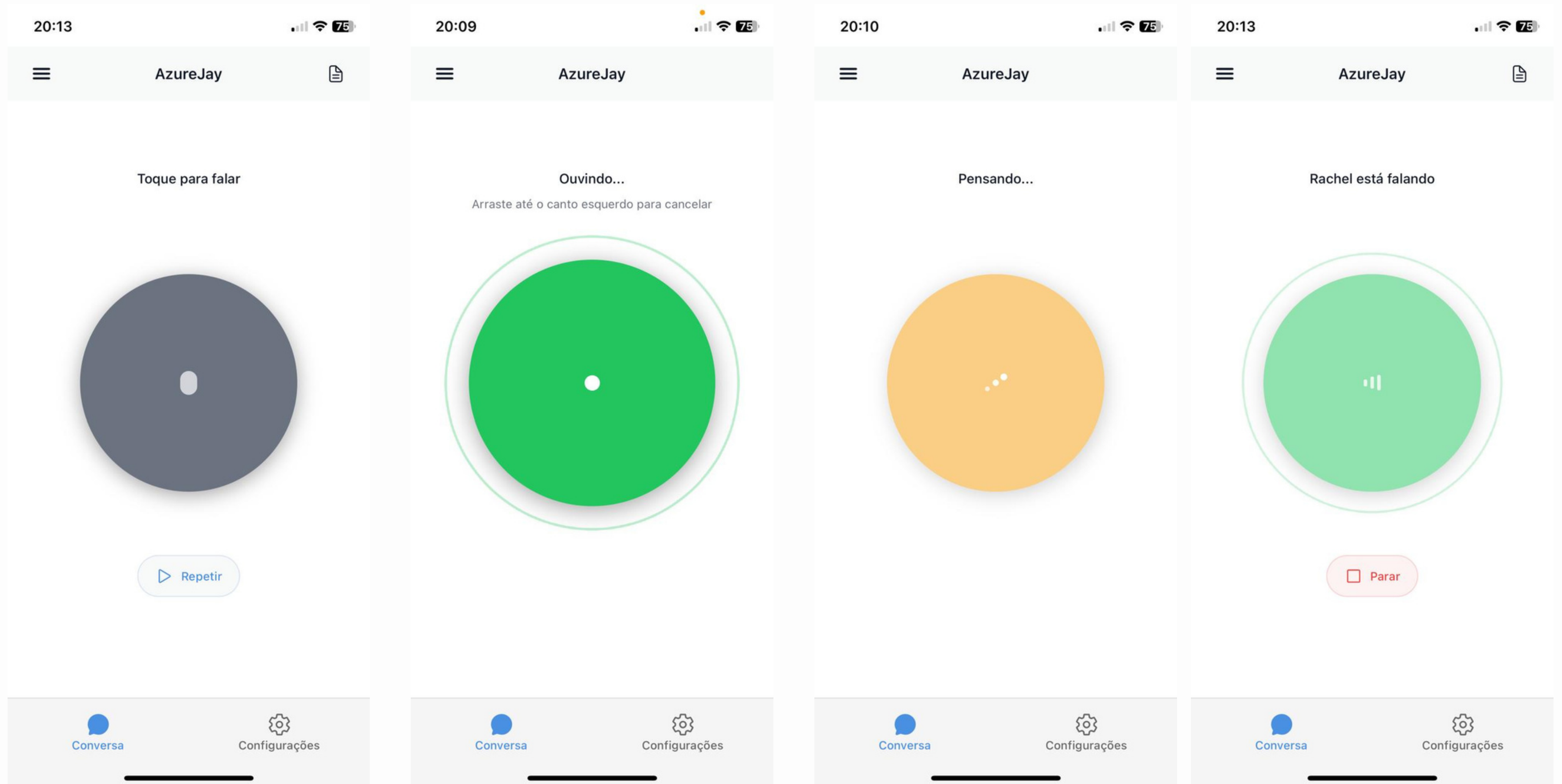
EXPERIÊNCIA DO USUÁRIO

Para uma boa experiência do usuário durante a interação por voz, é importante o retorno imediato do estado da reprodução do áudio (em reprodução, pausa, reproduzido, etc).

Estados de áudio

O botão central de áudio exibe estados de reprodução distintos visualmente para cada fase da interação.

1. ***Gravando (ouvindo).***
2. ***Processando (Pensando).***
3. ***Reproduzindo (Executando o áudio).***



Fonte: Autoria própria.

FEEDBACK PEDAGÓGICO

Fornece a sugestão de correção para soar de forma mais nativa através da tela de transcrição.

Implementação

1. *O componente MessageItem é responsável por verificar os dados de cada mensagem recebida da API.*
2. *No servidor, a entidade GrammarCorrection armazena a análise da correção.*
3. *Quando o cliente recebe o histórico, o MessageItem verifica a presença do objeto correction.*
 - a. *Se correction existe, ele exibe a sugestão de correção em destaque, diretamente abaixo da mensagem original do usuário.*

×

Transcrição

Hello, how is you? My name is Luis and I...

Você

Hello, how is you? My name is Luis and I really love rugby. Can you tell me the result of the last Crusaders game?

SUGESTÃO

"Use 'are' for plural subjects"

Subject-verb agreement error

Rachel AI

Hey Luis, great to meet you! I noticed you said "how is you" - just a friendly tip, we usually say "how are you" because "you" is a plural subject. Anyway, I saw you're a big rugby fan, especially the Crusaders! Their last game was against the Force, and they won 55-33. How do you think they'll perform in their next match?

Continuar Conversa

Conversa

Configurações

×

Transcrição

Hey Rachel, what exactly is the rule of h...

Você

Hey Rachel, what exactly is the rule of holding on in the tackle?

Rachel AI

Hey Luis, so it seems like when it comes to holding onto the ball in a tackle, the key is to protect it by keeping it close to your chest. As for the tackler, they can use their legs to bring the player down, as long as they're holding onto the player and not just the ball. Does that make sense to you? What's your favorite position to play in rugby, by the way?

2 mensagens

0 correções

Continuar Conversa

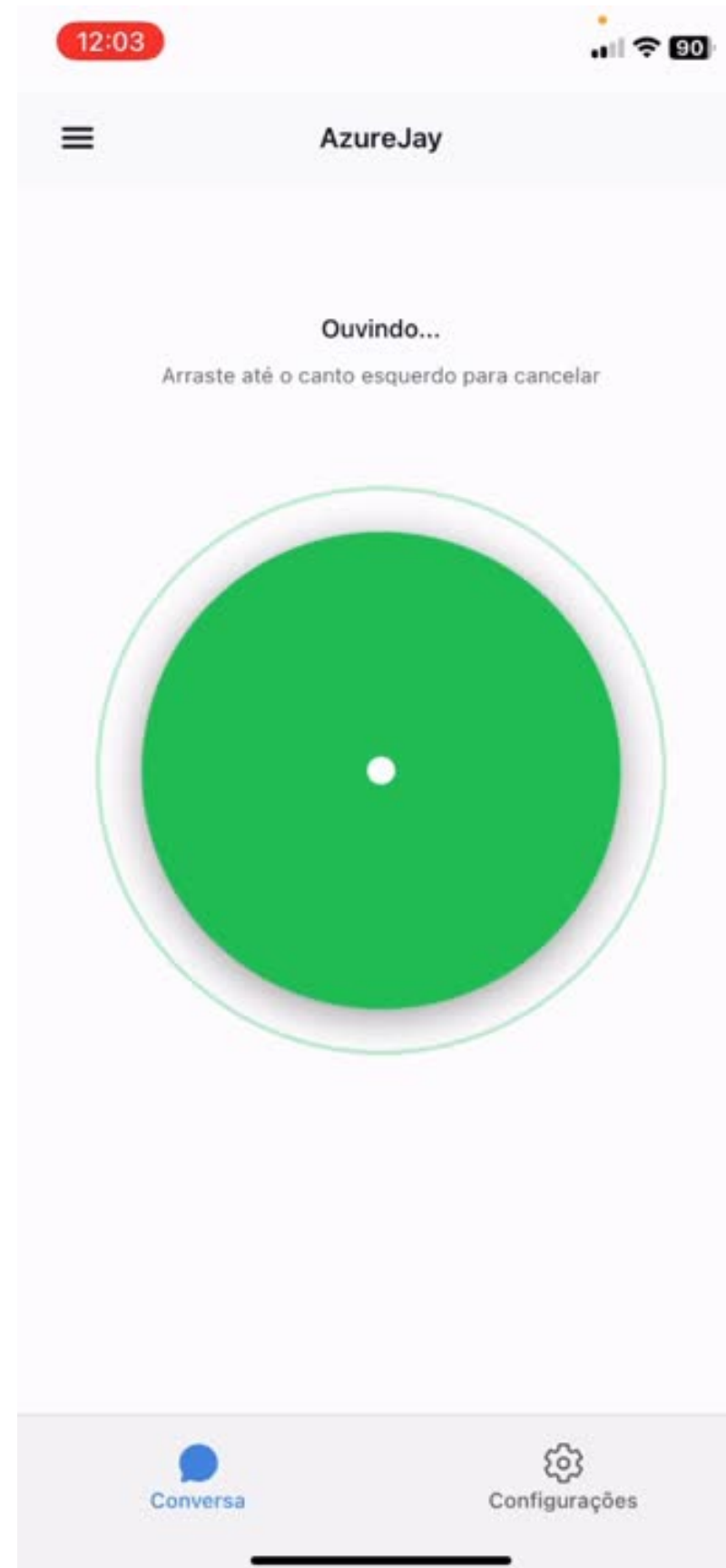
Conversa

Configurações

Fonte: Aatoria própria.

Materiais e Métodos

58



VÍDEO DE APRESENTAÇÃO



[Link de acesso para o vídeo.](#)

RESULTADOS

AVALIAÇÃO

A análise foi feita tanto quantitativamente quanto qualitativamente.

Análise qualitativa

Com o objetivo de discutir a eficácia da arquitetura do agente, foram adotados os principais pilares do framework OAgents (Zhu et al., 2025), um estudo que define os componentes para um agente inteligente efetivo.

Análise quantitativa

Com o objetivo de medir a eficiência (custos e latência) foi feito um teste do pior cenário comparando a arquitetura proposta (AFM/CoA) com uma arquitetura multiagente tradicional.

ANÁLISE QUALITATIVA

A arquitetura proposta atende aos 3 pilares mais fundamentais de um agente efetivo (Zhu et al., 2025).

Capacidade de planejamento

- O agente implementa planejamento explícito (via tag <plan>) e decomposição de subtarefas.
- A tag <reflection> serve como um mecanismo de revisão dinâmica do plano e autocrítica, permitindo ao agente ajustar sua estratégia.

Utilização de ferramentas

- O agente invoca as ferramentas necessárias conforme o plano estabelecido.

Gerenciamento de memória

Memória de curto prazo: Implementada passando o histórico da conversa atual para o prompt a cada turno, servindo como memória de trabalho.

Memória de longo prazo: Implementada pela ferramenta UpdateUserProfile e o padrão Write-Through Cache (Redis + PostgreSQL), garantindo acesso rápido e persistente ao perfil do usuário.

ANÁLISE QUANTITATIVA

Teve como propósito quantificar o custo (chamadas de API) e a latência (tempo) da arquitetura proposta.

Ambiente de teste

- A aplicação foi containerizada (Docker) e executada em ambiente local (Dell G15, Core i5, 16GB RAM).

Componente	Especificação
Máquina (Host)	
Sistema	Notebook Dell G15 5511
Processador (CPU)	Intel Core i5 (9ª Geração) @ 2.69 GHz
Memória (RAM)	16 GB
Sistema Operacional	Windows 11 Home Single Language (x64)
Ambiente de execução	
Imagens Docker	Python 3.11-slim, Postgres 17, Redis Alpine

Fonte: Elaborado pelo autor.

Teste realizado (pior cenário)

- Foi utilizado um prompt projetado para acionar todos os recursos do agente de uma só vez (correção gramatical, atualização de memória e busca na web):
 - "Hello, my name is Luis and I really loves Rugby. Can you tell me the result of the last Crusaders game?"

RESULTADOS DA ANÁLISE QUANTITATIVA

Chamadas de API (Custo)

- A arquitetura proposta (AFM) reduziu o número de chamadas ao LLM em 71,43% (caindo de 28 para 8).

O paradigma CoA internaliza o raciocínio (planejamento, crítica) que, na abordagem multiagente, gasta chamadas de API apenas para a correta comunicação entre os agentes.

Tempo de resposta (Latência)

- Como consequência direta, a latência total foi reduzida em 72,71% (caindo de 32,21s para 8,79s).

CONCLUSÃO

CONSIDERAÇÕES FINAIS

AGENTES MODERNOS NO APRENDIZADO DE NOVAS LÍNGUAS

Abordagens puramente neurais

Superam a imprecisão das abordagens puramente neurais (LLMs) ao permitir a inserção de conhecimento estruturado.

Paradigma CoA x Multiagente

O paradigma CoA, mesmo implementado parcialmente, foi validado como uma solução viável, superando a alta latência e custos das arquiteturas multiagente.

Aplicação pedagógica

O aplicativo móvel supera o aprendizado por computadores, oferecendo um ambiente seguro para a prática de conversação com feedback instantâneo.

LIMITAÇÕES

É fundamental salientar que a aplicação possui caráter acadêmico, sendo um protótipo, não sendo desenvolvida para substituir a instrução formal de um professor qualificado, mas sim como uma ferramenta de prática adicional.

01

VALIDAÇÃO DE DESEMPENHO

Embora a análise quantitativa tenha validado a eficiência da arquitetura (redução de 72,71% na latência), o trabalho não se estendeu a uma avaliação em benchmarks padronizados.

02

IMPLEMENTAÇÃO DO COA

O fluxo do AFM foi realizado via engenharia de prompt (in-context learning), e não pelo processo formal de destilação de conhecimento (*multi-agent distillation*), que otimizaria ainda mais a eficiência.

TRABALHOS FUTUROS

A principal evolução seria implementar o processo de destilação de conhecimento (multi-agent distillation). Isso criaria um AFM verdadeiramente especializado, superando a engenharia de prompt atual.

01

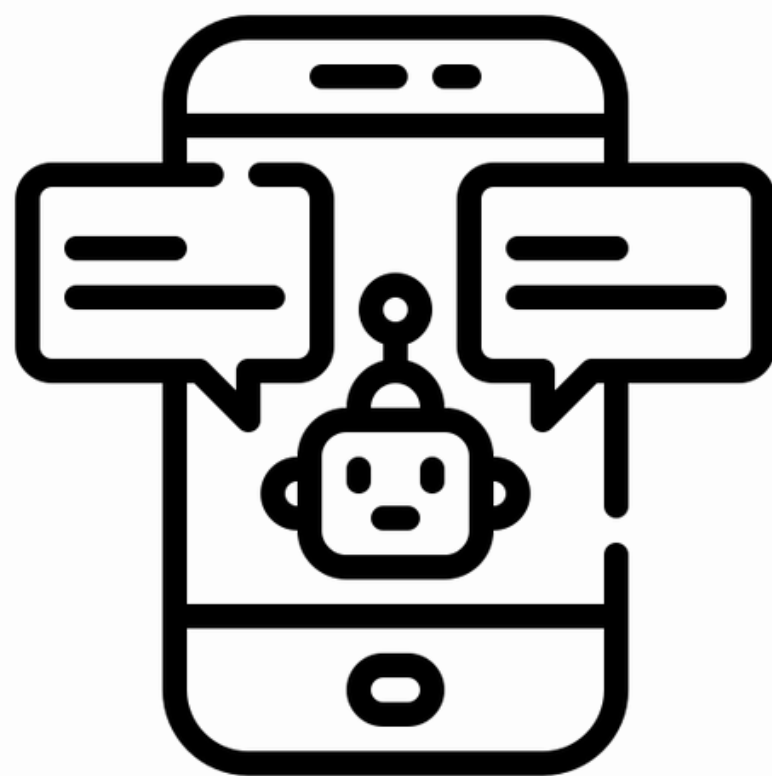
VALIDAÇÃO DE DESEMPENHO

Expandir a validação quantitativa, analisando a performance em benchmarks padronizados para uma comparação mais ampla com outras arquiteturas.

02

VALIDAÇÃO PEDAGÓGICA

Conduzir estudos de caso com usuários reais e obter feedback de professores qualificados na área do ensino de inglês, com o intuito de aprimorar os aspectos pedagógicos da aplicação.



REFERÊNCIAS

BERNACKI, M. L.; GREENE, J. A.; CROMPTON, H. Mobile technology, learning, and achievement: Advances in understanding and measuring the role of mobile technology in education. *Contemporary Educational Psychology*, Elsevier, v. 60, . 101827, 2020.

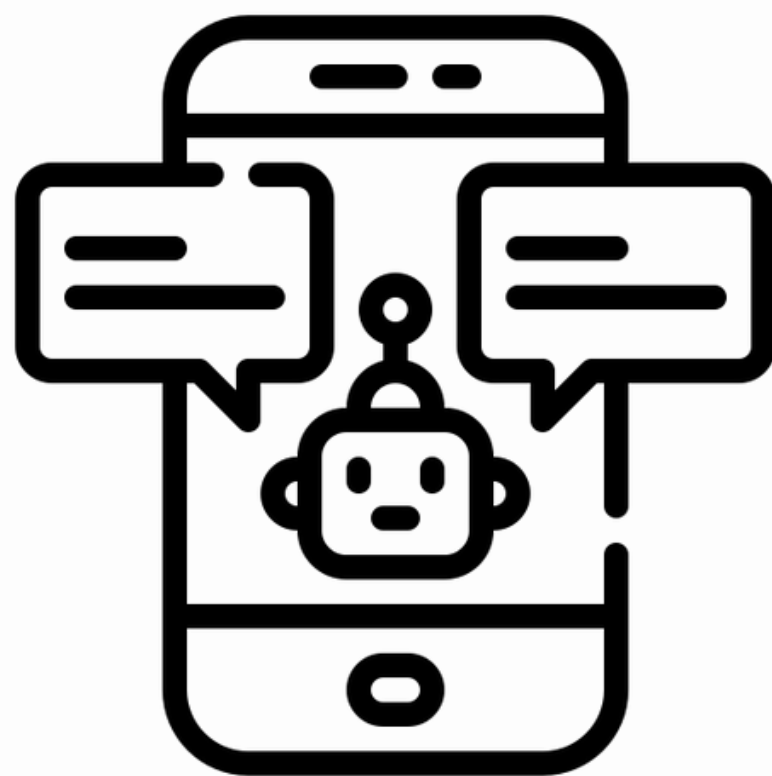
BUCHNER, J.; ANDUJAR, A. The expansion of the classroom through mobile immersive learning. *International Association for Development of the Information Society*, ERIC, 2019.

CASELI, H. M.; NUNES, M. G. V. (Ed.). *Processamento de Linguagem Natural: Conceitos, Técnicas e Aplicações em Português*. 3. ed. BPLN, 2024.

ESCUETA, M.; QUAN, V.; NICKOW, A. J.; OREOPOULOS, P. Education technology: An evidence-based review. *National Bureau of Economic Research*, 2017.

FUKSA, M.; SPETH, S.; BECKER, S. Mvvm revisited: Exploring design variants of the model-view-viewmodel pattern. In: SPRINGER. *International Conference on Enterprise Design, Operations, and Computing*. [S.I.], 2024. p. 163-181.

HADFIELD, J.; ZHANG, B.; LIEN, K.; SCHOLZ, F.; FOX, J.; FORD, D. How we built our multi-agent research system. 2024. Disponível em: <https://www.anthropic.com/engineering/multi-agent-research-system/>. Acesso em: 05 nov. 2025.



REFERÊNCIAS

HORWITZ, E. K.; HORWITZ, M. B.; COPE, J. Foreign language classroom anxiety. The Modern language journal, JSTOR, v. 70, n. 2, p. 125-132, 1986.

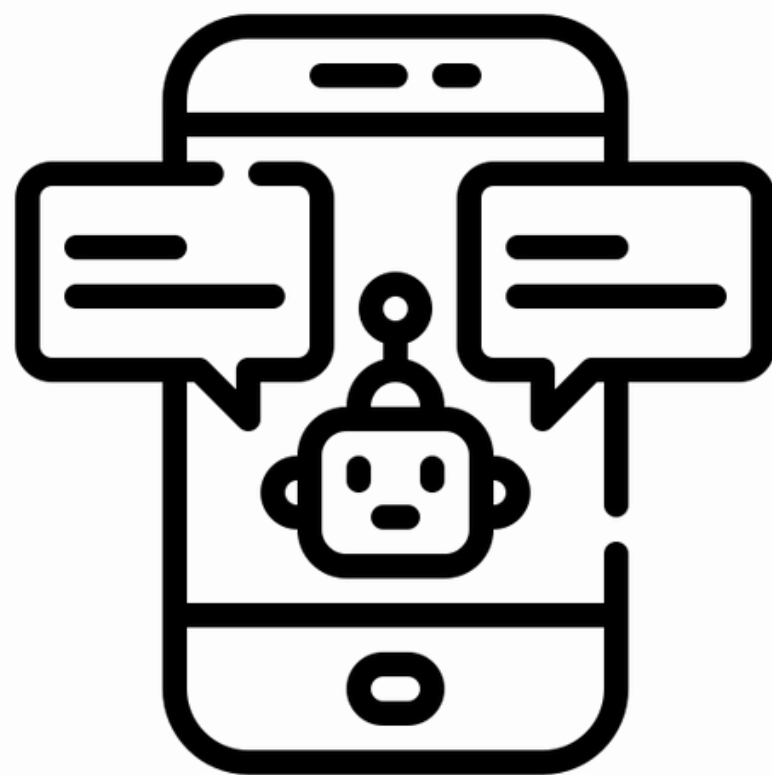
IBGE, A. Pesquisa IBGE 2023. 2024. Disponível em: <https://agenciadenoticias.ibge.gov.br/agencia-noticias/2012-agencia-de-noticias/noticias/41026-em-2023-87-2-das-pessoas-com-10-anos-ou-mais-utilizaram-internet/>. Acesso em: 08 nov. 2025.

ILYOSOVNA, N. A. The importance of english language. International Journal on Orange Technologies, Research Parks Publishing, v. 2, n. 1, p. 22-24, 2020.

JOUPPI, N. P. Cache write policies and performance. ACM SIGARCH Computer Architecture News, ACM New York, NY, USA, v. 21, n. 2, p. 191-201, 1993.

KARPAS, E.; ABEND, O.; BELINKOV, Y.; LENZ, B.; LIEBER, O.; RATNER, N.; SHOHAM, Y.; BATA, H.; LEVINE, Y.; LEYTON-BROWN, K. et al. Mrkl systems: A modular, neuro-symbolic architecture that combines large language models, external knowledge sources and discrete reasoning. arXiv preprint arXiv:2205.00445, 2022.

LEVY, M. Computer-assisted language learning: Context and conceptualization. [S.l.]: Oxford university press, 1997.



REFERÊNCIAS

LI, W.; LIN, J.; JIANG, Z.; CAO, J.; LIU, X.; ZHANG, J.; HUANG, Z.; CHEN, Q.; SUN, W.; WANG, Q. et al. Chain-of-agents: End-to-end agent foundation models via multi-agent distillation and agentic rl. arXiv preprint arXiv:2508.13167, 2025.

LUBANOVIC, B. FastAPI. [S.I.]: "O'Reilly Media, Inc.", 2023.

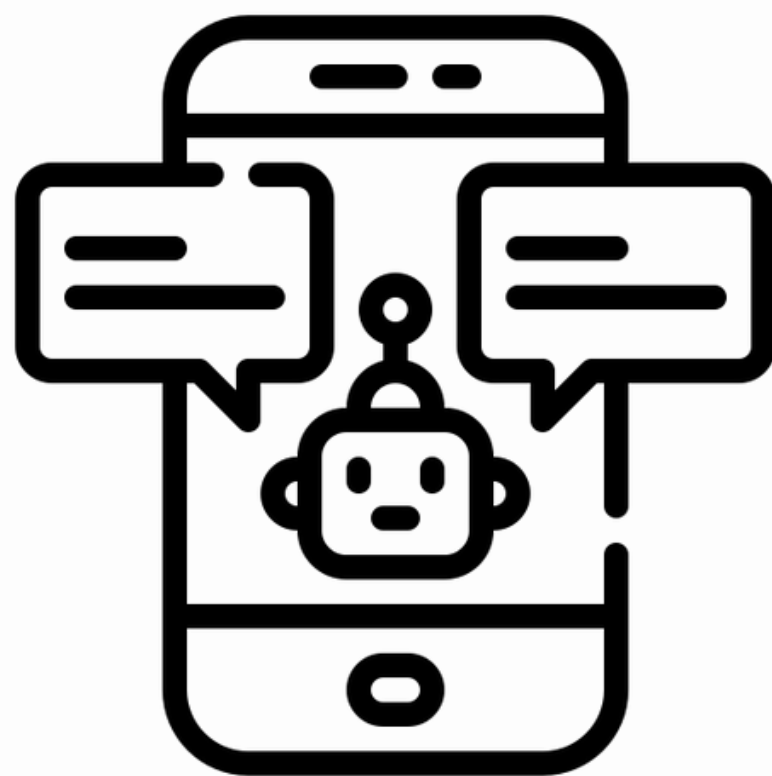
LYU, B.; LAI, C.; GUO, J. Effectiveness of chatbots in improving language learning: A meta-analysis of comparative studies. International Journal of Applied Linguistics, Wiley Online Library, v. 35, n. 2, p. 834-851, 2025.

MARCUS, G. Deep learning: A critical appraisal. arXiv preprint arXiv:1801.00631, 2018.

MARTIN, R. C. Clean architecture: a craftsman's guide to software structure and design. [S.I.]: Prentice Hall Press, 2017.

MIRA, J. M. Symbols versus connections: 50 years of artificial intelligence. Neurocomputing, Elsevier, v. 71, n. 4-6, p. 671-680, 2008.

PELLURU, K. Langchain & langgraph in production: Architectures for multi-agent llm systems. Journal of Data and Digital Innovation (JDDI), v. 2, n. 3, p. 1-9, 2025.



REFERÊNCIAS

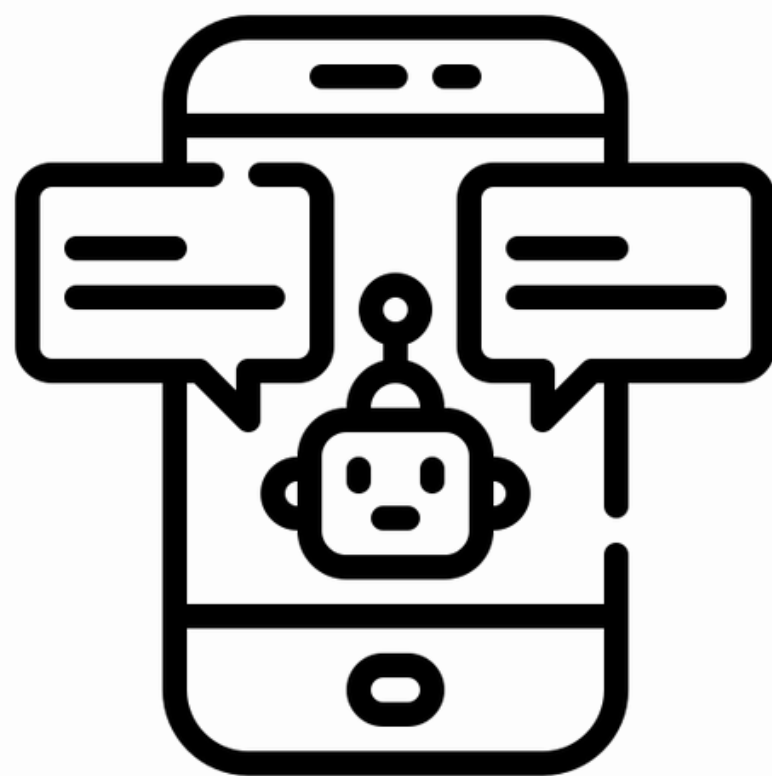
SHAHZAD, T.; MAZHAR, T.; TARIQ, M. U.; AHMAD, W.; OUAHADA, K.; HAMAM, H. A comprehensive review of large language models: issues and solutions in learning environments. *Discover Sustainability*, Springer, v. 6, n. 1, p. 27, 2025.

SHORTT, M.; TILAK, S.; KUZNETCOVA, I.; MARTENS, B.; AKINKUOLIE, B. Gamification in mobile-assisted language learning: A systematic review of duolingo literature from public release of 2012 to early 2020. *Computer Assisted Language Learning*, Taylor & Francis, v. 36, n. 3, p. 517-554, 2023.

VALENTE, M. T. Engenharia de software moderna. Princípios e práticas para desenvolvimento de software com produtividade, v. 1, n. 24, 2020.

WEI, J.; WANG, X.; SCHUURMANS, D.; BOSMA, M.; XIA, F.; CHI, E.; LE, Q. V.; ZHOU, D. et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, v. 35, . 24824-24837, 2022.

YAO, S.; ZHAO, J.; YU, D.; DU, N.; SHAFRAN, I.; NARASIMHAN, K.; CAO, Y. React: Synergizing reasoning and acting in language models. In: *International Conference on Learning Representations (ICLR)*. [S.l.: s.n.], 2023.



REFERÊNCIAS

YU, D.; YANG, B.; LIU, D.; WANG, H.; PAN, S. A survey on neural-symbolic learning systems. Neural Networks, Elsevier, v. 166, p. 105-126, 2023.

ZHANG, Z.; HUANG, X. The impact of chatbots based on large language models on second language vocabulary acquisition. Heliyon, Elsevier, v. 10, n. 3, 2024.

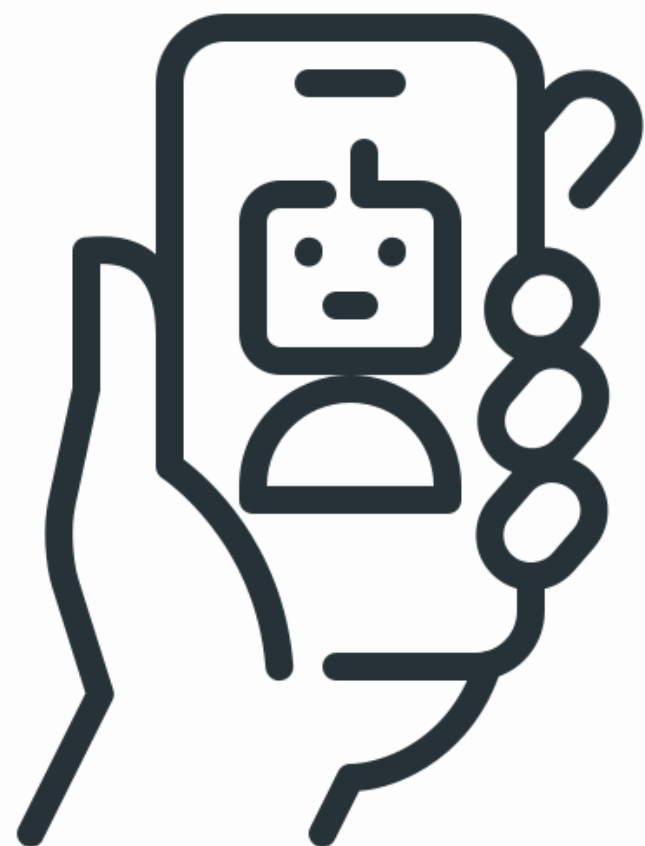
ZHU, H.; QIN, T.; ZHU, K.; HUANG, H.; GUAN, Y.; XIA, J.; YAO, Y.; LI, H.; WANG, N.; LIU, P. et al. Oagents: An empirical study of building effective agents. arXiv preprint arXiv:2506.15741, 2025.

ARUN SHANKAR. Building ReAct Agents from Scratch. 2024. Disponível em: <https://medium.com/google-cloud/building-react-agents-from-scratch-a-hands-on-guide-using-gemini-ffe4621d90ae>. Acesso em: 08 nov. 2025.

SANDER SCHULHOFF. MRKL Systems. 2023. Disponível em: <https://learnprompting.org/docs/agents/mrk>. Acesso em: 08 nov. 2025.

CARLSON, J. Redis in action. [S.l.]: Simon and Schuster, 2013.

EXPO. Expo Documentation. 2025. Disponível em: <https://docs.expo.dev/>. Acesso em: 10 nov. 2025.



O B R I G A D O .